

Conception orientée objet avec UML



EL AKEL BOUCHRA

Sommaire

- ❑ **Notions de modèle et de modélisation**
- ❑ **L'approche Orientée objet**
- ❑ **Le langage de modélisation UML**

Notions de modèle / modélisation

Qu'est ce qu'un modèle ?

Un modèle est une **représentation abstraite** et simplifiée (i.e. qui exclut certains détails), d'une entité (phénomène, processus, système, etc.) du monde réel en vue de le décrire, de l'expliquer ou de le prévoir.

Concrètement, un modèle permet de réduire la **complexité** d'un phénomène en éliminant les détails qui n'influencent pas son comportement de manière significative.

Pourquoi modéliser ?

- Modéliser un système avant sa réalisation permet de mieux comprendre le fonctionnement du système.
- C'est également un bon moyen de maîtriser sa complexité et d'assurer sa cohérence.
- Un modèle est un langage commun, précis, qui est connu par tous les membres de l'équipe et il est donc, à ce titre, un vecteur privilégié pour communiquer.
- Cette communication est essentielle pour aboutir à une compréhension commune aux différentes parties prenantes et précise d'un problème donné.
- Dans le domaine de l'ingénierie du logiciel, le modèle permet de mieux répartir les tâches et d'automatiser certaines d'entre elles.
- C'est également un facteur de réduction des coûts et des délais (génération de code par les plateformes de modélisation).

Les intervenants d'un projet informatique

Les intervenants d'un projet informatique (1/2)

Le client

Le client est l'organisme auquel est destiné le projet, c'est celui le donneur d'ordre et le payeur de la prestation.

Le client peut être une entreprise qui fait appel à une SSII (Société de services et d'ingénierie en informatique) pour réaliser le projet ou le service de l'entreprise qui fait appel à la direction informatique

Le prestataire

Le prestataire est l'organisme qui réalise le projet .

Le prestataire peut être une entreprise externe spécialisée (SSII) ou le service informatique de l'entreprise

Le directeur du projet ou chef de projet

Le Directeur ou chef de projet est le responsable de la mise en œuvre du projet dans le cadre du cahier des charges établi.

Les intervenants d'un projet informatique (2/2)

Le responsable qualité

Le responsable qualité est choisi en commun accord entre le maître d'oeuvre et le maître d'ouvrage . Son rôle :

- Définir les dispositions d'assurance qualité formalisées dans le plan d'assurance qualité.
- Veiller à la mise en application de ces dispositions.
- Définir les actions correctives si les dispositions ne sont appliquées.
- Vérifier et rendre compte de la mise en application de ces actions.

Les « utilisateurs »

Les utilisateurs sont les destinataires finaux du projet. Ils participent au projet sous la responsabilité du maître d'ouvrage.

Le rôle des utilisateurs est important en particulier au niveau de :

- L'expression des besoins.
- Les tests de recette.
- La mise en service du projet.

UML : **Unified Modeling Language**

UML (Unified Modeling Language)

L'OMG (Object Management Group) adopte en novembre 1997 UML 1.1 comme **langage de modélisation des systèmes d'information à objets**.

UML est donc non seulement un outil intéressant mais une norme qui s'impose en technologie à objets et à laquelle se sont rangés tous les grands acteurs du domaine, acteurs qui ont d'ailleurs contribué à son élaboration.

UML est un Langage de Modélisation Unifié

Langage : support de communication, avec une grammaire, *comme tout langage, il permet :*

- ☐ *de communiquer, de se faire comprendre*
- ☐ *de réfléchir et développer sa pensée*

Modélisation : description d'un système et abstraction

UML (Unified Modeling Language)

La version d'UML en cours à la fin 2006 est UML 2.0 et les travaux d'amélioration se poursuivent.

La dernière version de la spécification validée par l'OMG est UML 2.5.1 (2017)

Les objectifs d'UML

- Représenter des systèmes entiers
- Créer un langage de modélisation utilisable à la fois par les humains et les machines

Recherche d'un langage commun :

- . Utilisable par toutes les méthodes
- . Adapté à toutes les phases du développement
- . Compatible avec toutes les techniques de réalisation

UML un langage pour

- . visualiser
 - chaque symbole graphique a une sémantique
- . spécifier
 - de manière précise et complète, sans ambiguïté,
- . construire
 - les classes, les relations SQL peuvent être générées automatiquement
- . documenter
 - les différents diagrammes, notes, contraintes, exigences seront présentés dans un document.

Diagrammes UML

UML comporte ainsi treize types de diagrammes représentant autant de vues distinctes pour représenter des concepts particuliers du système d'information. Ils se répartissent en 3 groupes :

Diagrammes structurels (statiques) : UML Structure

- ❑ diagramme de **classes** (Class diagram)
- ❑ diagramme d'objets (Object diagram)
- ❑ diagramme de composants (Component diagram)
- ❑ diagramme de déploiement (Deployment diagram)
- ❑ diagramme de **paquetages** (Package diagram)
- ❑ diagramme de structures composites (Composite structure diagram)

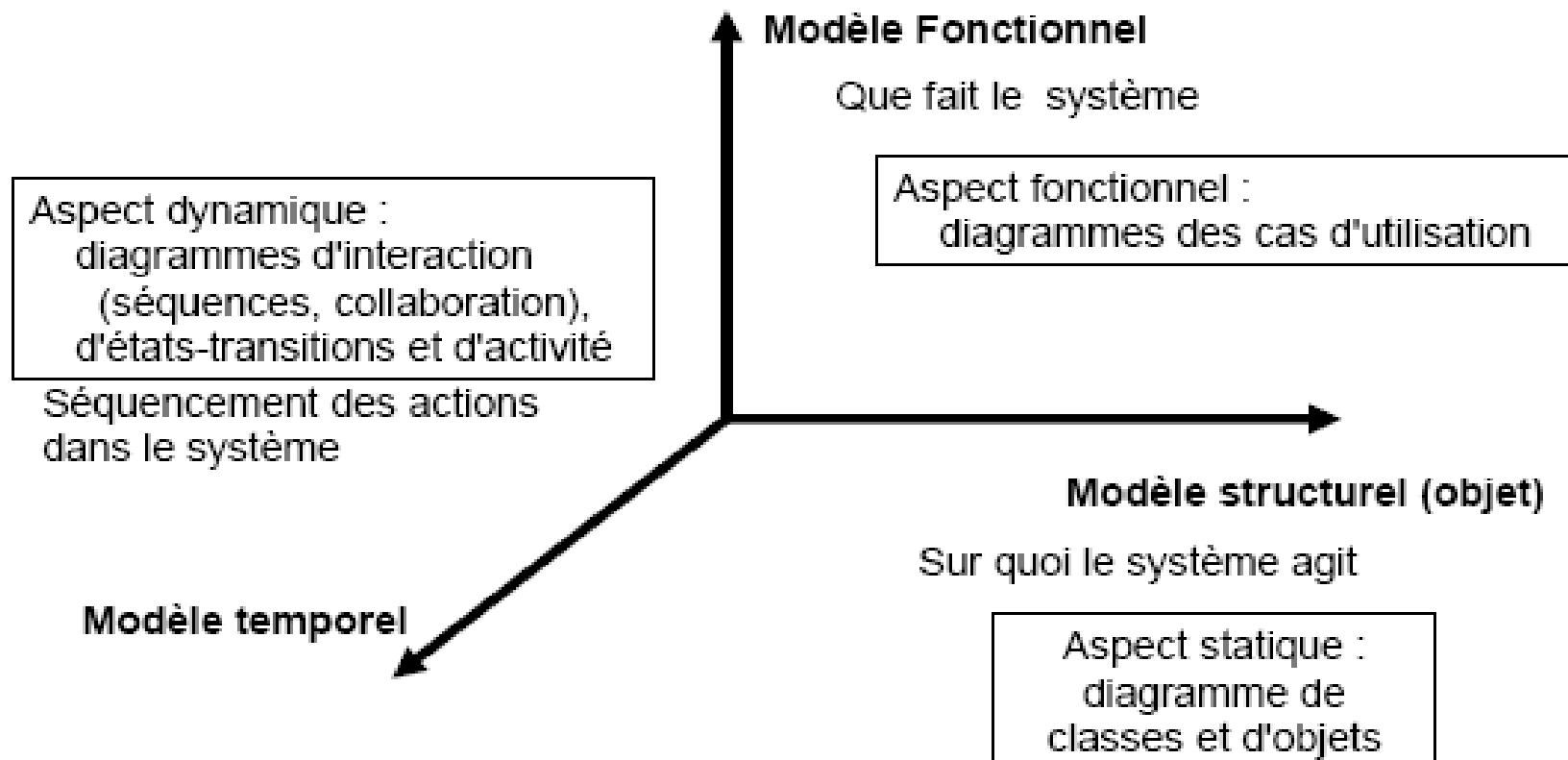
Diagrammes comportementaux (UML Behavior)

- ❑ diagramme de **cas d'utilisation** (Use case diagram)
- ❑ diagramme d'**activités** (Activity diagram)
- ❑ diagramme d'**états-transitions** (State machine diagram)

Diagrammes d'interaction ou dynamiques

- ❑ diagramme de **séquence** (Sequence diagram)
- ❑ diagramme de communication (Communication diagram)
- ❑ diagramme global d'interaction (Interaction overview diagram)
- ❑ diagramme de temps (Timing diagram)

Les trois composantes d'une modélisation



Phases de la modélisation

Cahier des charges

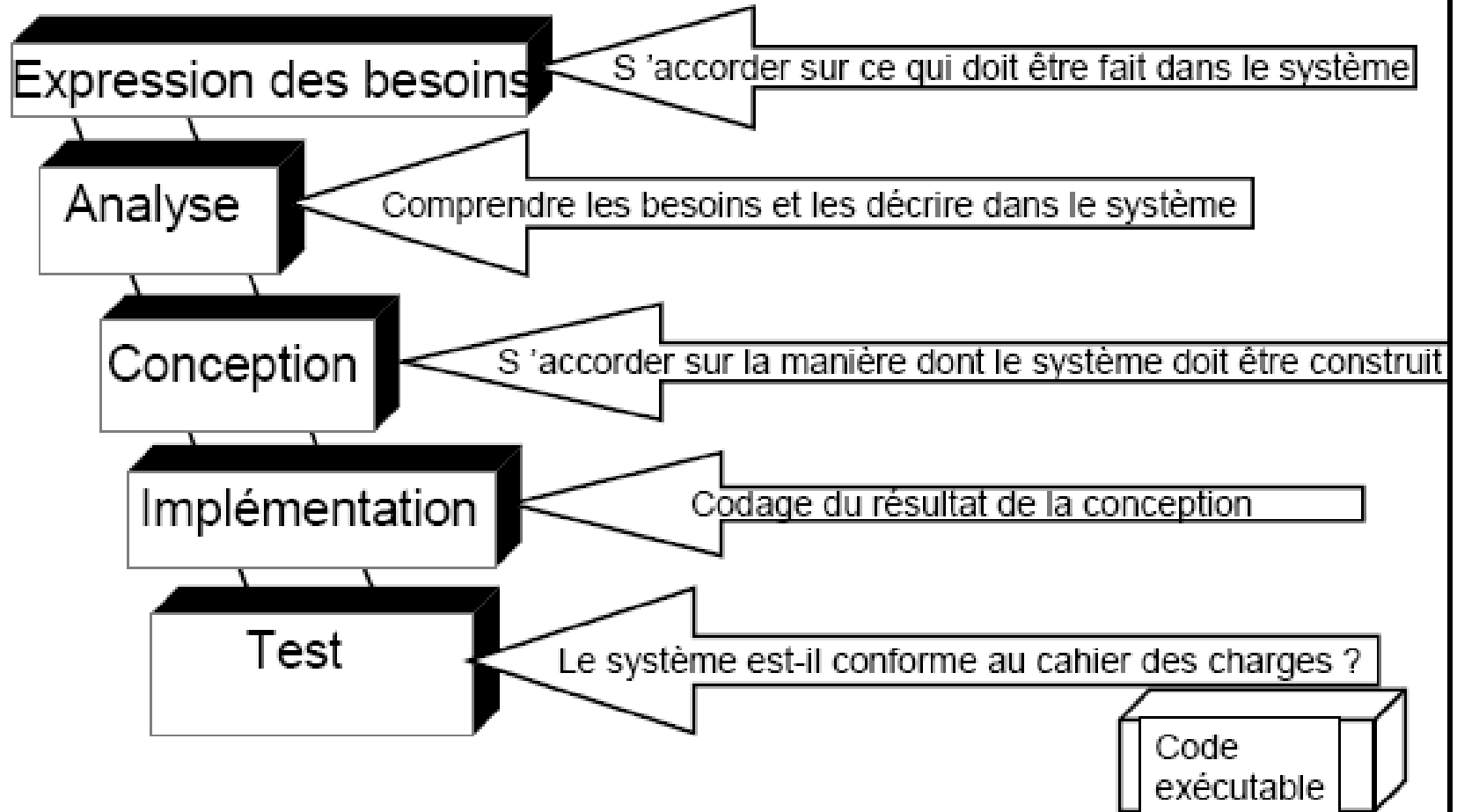


Diagramme des cas d'utilisation

Qu'est-ce qu'un diagramme de cas d'utilisation ?

Le **diagramme de cas d'utilisation** (*Use Case Diagram*) est un diagramme UML qui permet de représenter :

- **qui utilise le système ;**
- **ce que ces utilisateurs peuvent faire** avec le système ;
- les **relations entre les utilisateurs et les fonctionnalités ;**
- certaines relations entre les fonctionnalités elles-mêmes.

Il répond principalement à la question : « **Qui fait quoi dans le système ?** »

Il décrit donc le système **du point de vue de l'utilisateur**, sans expliquer comment le système est programmé.

Qu'est-ce qu'un diagramme de cas d'utilisation ?

Exemple:

Pour une application de gestion d'une bibliothèque :

- Un **bibliothécaire** peut ajouter un livre.
- Un **bibliothécaire** peut supprimer un livre.
- Un **adhérent** peut rechercher un livre.
- Un **adhérent** peut emprunter un livre.

Le diagramme de cas d'utilisation va représenter graphiquement ces interactions.

Pourquoi utiliser un diagramme de cas d'utilisation ?

Il sert principalement à :

Avant le développement

Identifier les fonctionnalités demandées : Qu'est-ce que l'application doit permettre de faire ?

Pour analyser les besoins

Comprendre les besoins des différents utilisateurs.

Pour communiquer avec le client

Le diagramme est relativement simple à comprendre même pour une personne qui ne connaît pas la programmation.

Pour préparer le développement

Les cas d'utilisation peuvent ensuite servir à définir :

- les fonctionnalités ;
- les user stories ;
- les interfaces ;
- les tests ;
- les scénarios fonctionnels.

diagramme de cas d'utilisation

Les éléments principaux

Un diagramme de cas d'utilisation comporte principalement :

- **Le système**
- **Les acteurs**
- **Les cas d'utilisation**
- **Les associations**
- **<<include>>**
- **<<extend>>**
- **La généralisation**

diagramme de cas d'utilisation

Le système

Le système représente l'application ou le logiciel que l'on souhaite développer.

On le représente généralement avec un **rectangle**.

Tout ce qui se trouve **à l'intérieur du rectangle** correspond aux fonctionnalités du système.

Les acteurs sont généralement placés **à l'extérieur**.

diagramme de cas d'utilisation

Acteur:

- Un **acteur** représente une personne, une organisation ou un autre système qui interagit avec notre système.
- Un acteur peut consulter et/ou modifier directement l'état du système, en émettant et/ou en recevant des messages éventuellement porteurs de données.

Les acteurs candidats sont systématiquement :

- les utilisateurs humains directs : identifiez tous les profils possibles, sans oublier l'administrateur, l'opérateur de maintenance, etc. ;
- les autres systèmes connexes qui interagissent aussi directement avec le système: Une application de commerce électronique peut communiquer avec un système bancaire.

diagramme de cas d'utilisation : Acteur

DISTINGUEZ L'ACTEUR PRINCIPAL DES ACTEURS SECONDAIRES

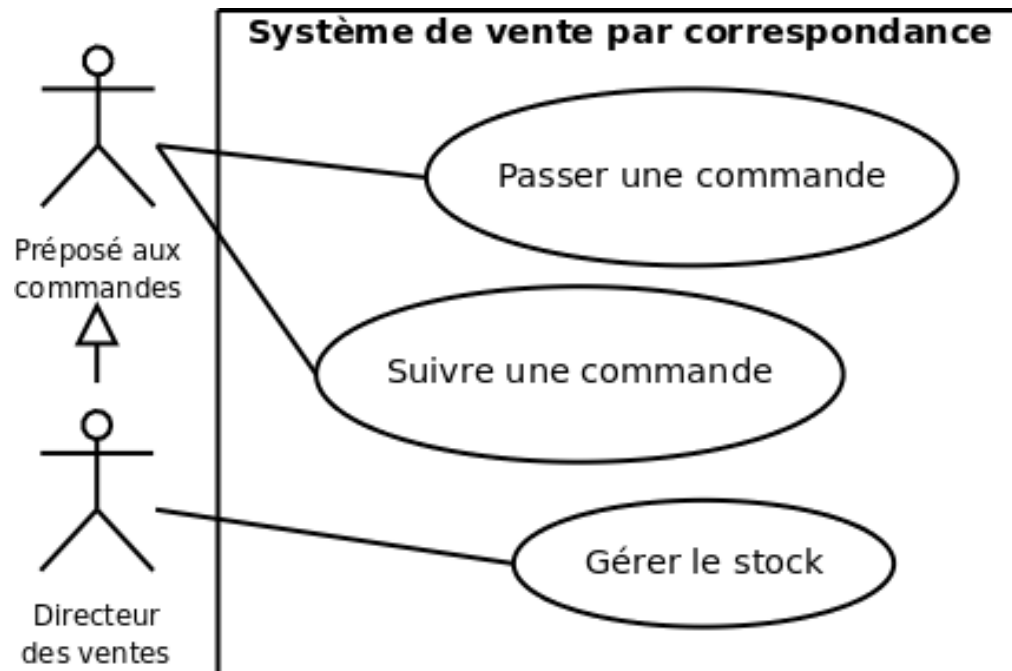
L'acteur principal est la plupart du temps le **déclencheur** du cas d'utilisation. Par opposition, nous qualifions d'acteurs secondaires les autres participants du cas d'utilisation. Les acteurs secondaires sont typiquement sollicités à leur tour par le système pour obtenir des informations complémentaires.

diagramme de cas d'utilisation : Relations entre les acteurs

- La seule relation possible entre deux acteurs est la généralisation : un acteur A est une généralisation d'un acteur B si l'acteur A peut être substitué par l'acteur B. Dans ce cas, tous les cas d'utilisation accessibles à A le sont aussi à B, mais l'inverse n'est pas vrai.
- Le symbole utilisé pour la généralisation entre acteurs est une flèche avec un trait plein dont la pointe est un triangle fermé désignant l'acteur le plus général (comme nous l'avons déjà vu pour la relation de généralisation entre cas d'utilisation).

diagramme de cas d'utilisation : Relations entre les acteurs

- Par exemple, la figure montre que le directeur des ventes est un préposé aux commandes avec un pouvoir supplémentaire : en plus de pouvoir passer et suivre une commande, il peut gérer le stock. Par contre, le préposé aux commandes ne peut pas gérer le stock.



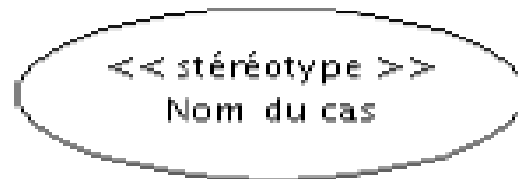
Elements d'un cas d'utilisation : le cas d'utilisation

Un cas d'utilisation est une unité cohérente d'une fonctionnalité visible de l'extérieur.

Il réalise un service de bout en bout, avec un déclenchement, un déroulement et une fin, pour l'acteur qui l'initie.

Un cas d'utilisation modélise donc un service rendu par le système, sans imposer le mode de réalisation de ce service.

Un cas d'utilisation se représente par une ellipse contenant le nom du cas (un verbe à l'infinitif).

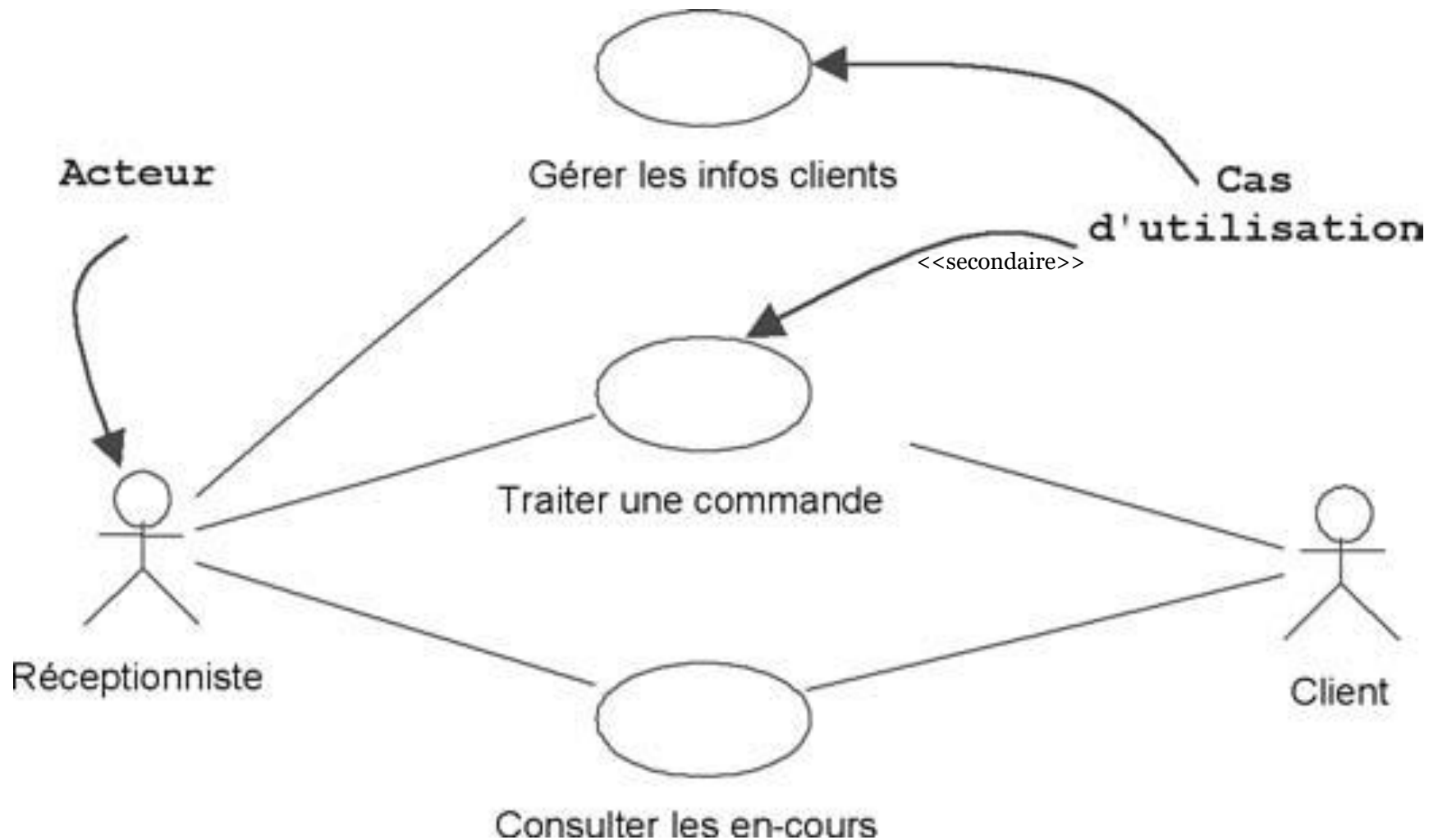


Les cas d'utilisation

NOMMEZ LES CAS D'UTILISATION AVEC UN VERBE À L'INFINITIF SUIVI D'UN COMPLÉMENT

Une recommandation complémentaire consiste à veiller à bien utiliser **le point de vue de l'acteur et non pas celui du système**. Par exemple, le cas d'utilisation d'un distributeur de billets par l'acteur « Porteur de CB » doit être intitulé « Retirer de l'argent » (point de vue de l'acteur), et non « Distribuer de l'argent » (point de vue du système).

Les cas d'utilisation : Exemple



Les cas d'utilisation : Exemple

Dans l'exemple précédent, nous avons trois cas de figure différents :

- *Gérer les infos clients, qui n'a qu'un seul acteur principal : le réceptionniste.*
- *Traiter une commande, qui a deux acteurs : le réceptionniste qui est principal, et le client qui est secondaire.*
- *Consulter les en-cours, qui a également deux acteurs. Tous deux peuvent accéder à la même fonctionnalité métier. Dans ce cas, l'acteur principal est celui qui tire réellement bénéfice des résultats du cas d'utilisation. Il s'agit en fait du client qui passe soit directement par Internet, soit par l'intermédiaire du réceptionniste pour obtenir l'information.*

Relations dans un cas d'utilisation

On distingue 3 types de relations entre les cas d'utilisation:

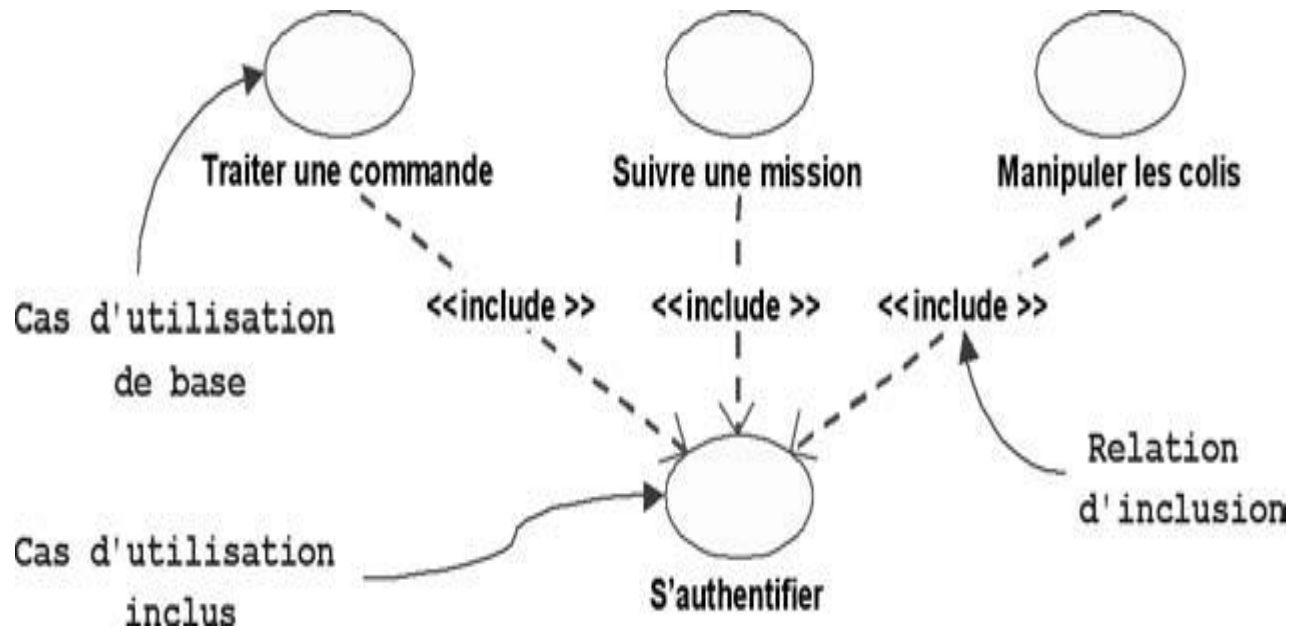
- ✓ Relation d'inclusion
- ✓ Relation d'extension
- ✓ Relation de généralisation

Relations dans un UC : Relation d'inclusion

Un cas A inclut un cas B si le comportement décrit par le cas A **inclut** le comportement du cas B : le cas A dépend de B. Lorsque A est sollicité, B l'est obligatoirement, comme une partie de A. Cette dépendance est symbolisée par le stéréotype « include ». Les inclusions permettent essentiellement de factoriser une partie de la description d'un cas d'utilisation qui serait commune à d'autres cas d'utilisation .

Représentée dans le diagramme de UC par le stéréotype : <<include>>

Relation d'inclusion : exemple



A include B : flèche vers B

Relations dans un UC : Relation d'extension

La relation d'extension est probablement la plus utile car elle a une sémantique qui a un sens du point de vue métier au contraire des deux autres qui sont plus des artifices d'informaticiens.

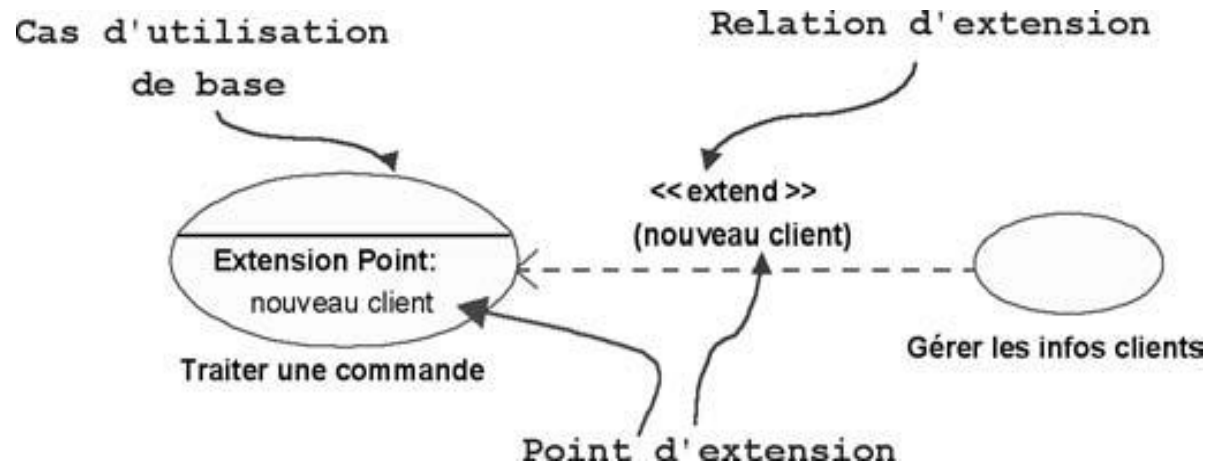
On dit qu'un cas d'utilisation A étend un cas d'utilisation B lorsque le cas d'utilisation A peut être appelé au cours de l'exécution du cas d'utilisation B.

Exécuter B peut éventuellement entraîner l'exécution de A .

L'extension peut intervenir à un point précis du cas étendu. Ce point s'appelle le point d'extension.

Il porte un nom, qui figure dans un compartiment du cas étendu sous la rubrique point d'extension, et est éventuellement associé à une contrainte indiquant le moment où l'extension intervient. Une extension est souvent soumise à condition. Graphiquement, la condition est exprimée sous la forme d'une note.

Relation d'extension : exemple



Dans le cas d'utilisation « Traiter une commande », un des enchaînements principaux consiste à créer une nouvelle commande. Or, si le client est inconnu du système, le réceptionniste va devoir interrompre son processus de création de commande pour tenter auparavant de créer un nouveau client. Si ce processus se déroule sans encombre, il pourra alors continuer sa création de commande. Le processus de création de client, pour sa part, fait partie intégrante du cas d'utilisation « Gérer les infos clients ». Il est donc intéressant de préciser cette relation d'extension entre les deux cas d'utilisation.

Relations dans un UC : Relation de généralisation

Un cas A est une généralisation d'un cas B si B est un cas particulier de A.

Cette relation de généralisation/spécialisation est présente dans la plupart des diagrammes UML et se traduit par le concept d'héritage dans les langages orientés objet.

Relation de généralisation : exemple

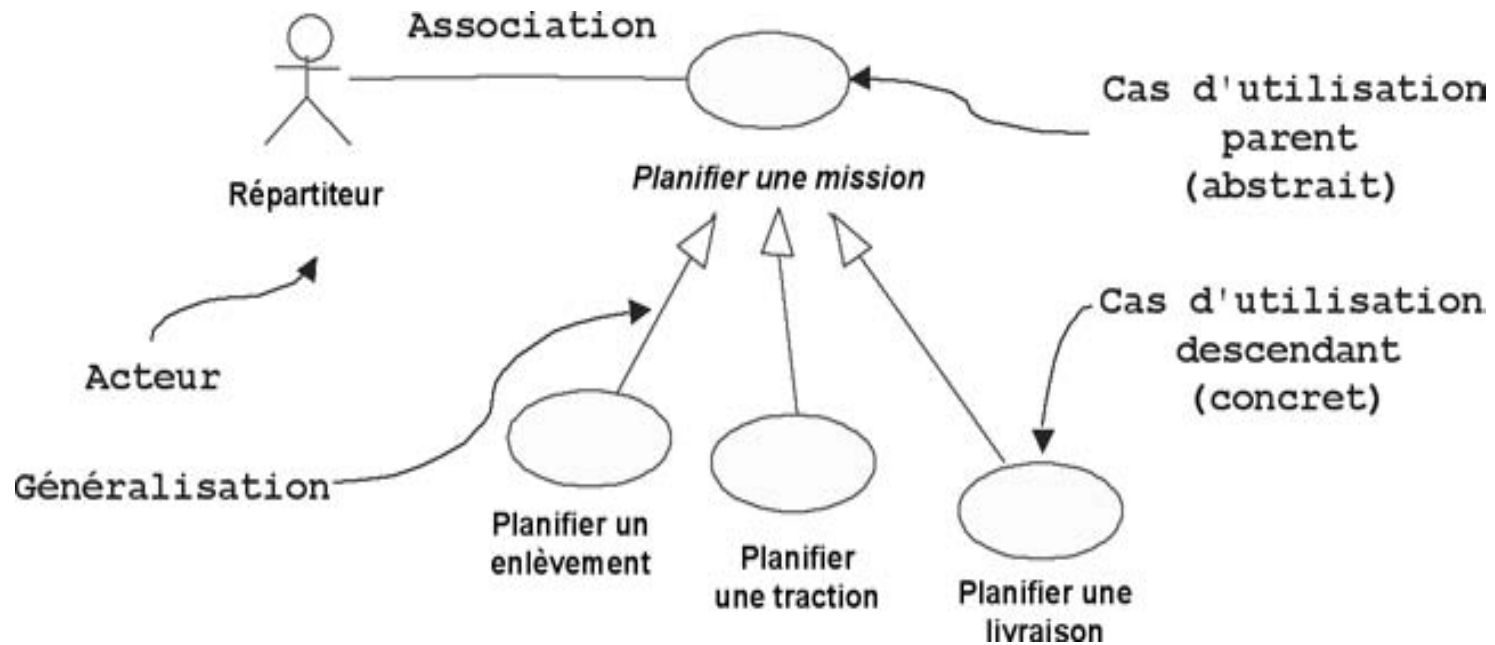
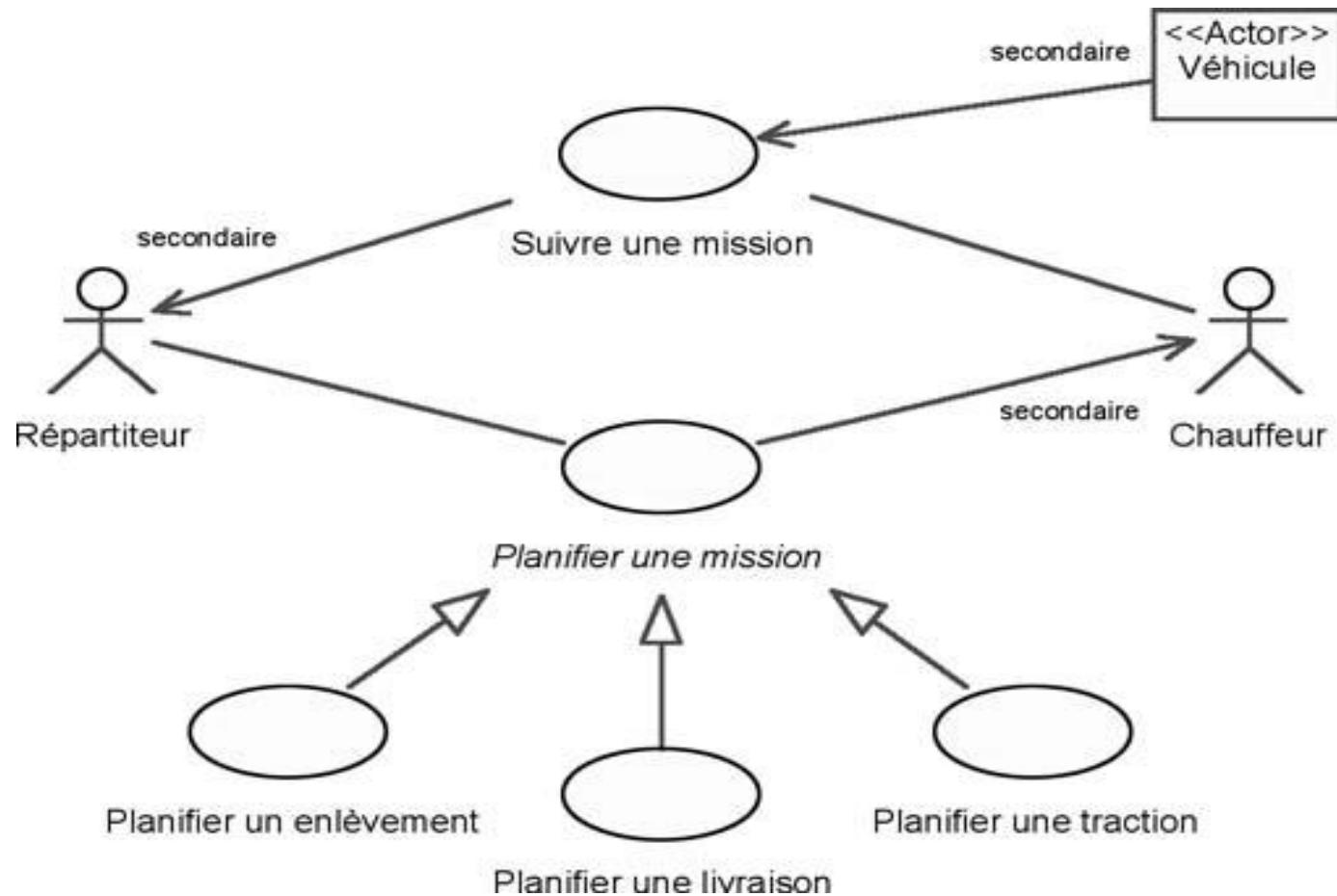


Diagramme de cas d'utilisation



Scénarios d'un cas d'utilisation

Scénarios d'un cas d'utilisation

- La description d'un cas d'utilisation se fait par des scénarios qui définissent la suite logique des interactions qui constituent ce cas.
- On peut définir des scénarios simples ou des scénarios plus détaillés faisant intervenir les variantes, les cas d'erreurs, etc.
- Cette description se fait de manière simple, par un texte compréhensible par les personnes du domaine de l'application.
- Elle précise ce que fait l'acteur et ce que fait le système
- La description détaillée pourra préciser les contraintes de l'acteur et celles du système.

Scénarios d'un cas d'utilisation

Réservation d'un livre

description simplifiée

Le client se présente devant un terminal:

- (1) Le système affiche un message d'accueil.
- (2) Le client choisit l'opération réservation parmi les différentes opérations proposées.
- (3) Le système lui demande de s'authentifier.
- (4) Le client donne son identification (nom, mot de passe).
- (5) Le système lui demande de choisir un livre.
- (6) Le client précise le livre qu'il désire.
- (7) Le système lui précise si un exemplaire du livre lui est réservé.

Scénarios d'un cas d'utilisation

Réservation d'un livre

description détaillée

Pré-conditions: Le client doit être inscrit à la bibliothèque

Le client ne doit pas avoir atteint le nombre maximum de
réservation

Un exemplaire du livre doit être enregistré

Post-conditions: (Si l'opération s'est bien déroulée)

Le client a une réservation supplémentaire

Le nombre d'exemplaires disponibles du livre est
décrémenté de 1

Scénarios d'un cas d'utilisation

Réservation d'un livre

Cas normal

description détaillée

1. (1) Le système affiche un message d'accueil sur le terminal avec un choix d'opérations
2. (2) Le client choisit l'opération réservation parmi les différentes opérations proposées.
3. (3) Le système lui demande de s'authentifier.
4. (4) Le client donne son identification (nom, mot de passe).
5. (5) Le système demande le titre du livre en donnant la possibilité de choisir dans une liste.
6. (6) Le client précise le livre qu'il désire.
7. (7) Le système lui précise que un exemplaire du livre lui est réservé.

variante 1

en (6) le client demande à connaître les livres présents

variante 2

en (4) le client n'est pas reconnu, dans ce cas, tant qu'il n'est pas reconnu, on lui redemande de s'authentifier

variante 3

en (4) le client est reconnu, mais le mot de passe est incorrect, il peut avoir 5 essais, par la suite le client sera interdit pendant 1 heure.

variante 4

en (4) le client n'a plus le droit de réserver

variante 5

en (7) le livre n'est pas libre

Scénarios d'un cas d'utilisation: Exemple

- Cas d'utilisation : Retirer de l'argent
- Acteur : Client
- Système : Guichet automatique bancaire

Étape	Acteur	Système
1	Insère sa carte bancaire	—
2	—	Demande le code PIN
3	Saisit son code PIN	—
4	—	Vérifie le code
5	Sélectionne « Retrait »	—
6	—	Demande le montant
7	Saisit 1 000 DH	—
8	—	Vérifie le solde
9	—	Distribue les billets
10	Récupère l'argent	—
11	—	Affiche « Retrait effectué avec succès »

Scénarios d'un cas d'utilisation: Exemple

- Cas d'utilisation: Commander un produit
- Acteur : Client

Système : Site e-commerce

Scénario:

- 1- **le client** recherche un produit.
- 2- **le Système** affiche les produits correspondants.
- 3- **le Client** sélectionne un produit.
- 4- **le Système** affiche les détails du produit.
- 5- **le Client** ajoute le produit au panier.
- 6- **le Système** affiche le panier.
- 7- **le Client** valide la commande.
- 8- **le Système** demande l'adresse de livraison.
- 9- **le Client** saisit son adresse.
- 10- **le Système** affiche le montant total.
- 11- **le Client** choisit le mode de paiement.
- 12- **le Système** confirme la commande.

Diagramme de séquence

Diagramme de séquence (1/2)

- Un scénario est une instance d'un cas d'utilisation
- Chaque cas d'utilisation a un WEB de scénario
- Les scénarios sont modélisés par les diagrammes d'interaction:
 - Diagrammes de séquence : Se basent sur l'aspect temporel
 - Diagrammes de collaboration (communication) : Montrent les flots de données

Diagramme de séquence (2/2)

- Les diagrammes de séquence montrent les interactions qui surviennent dans une séquence de temps d'un scénario donné.
- Les diagrammes des séquences documentent les interactions à mettre en œuvre entre les classes pour réaliser un résultat, tel qu'un cas d'utilisation.

Caractéristiques d'un diagramme de séquence:

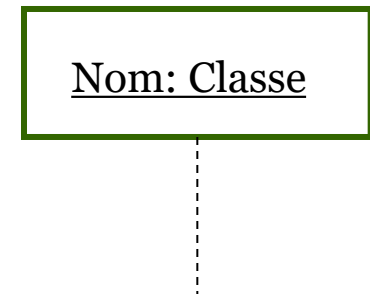
- Dimension verticale qui représente le temps ainsi que la durée de vie des objets
- Dimension horizontale qui représente les différents objets
- Les messages échangés entre les objets sont représentés horizontalement par une ligne fléchée.

Représentation (1/2)

Dans un diagramme des séquences, les classes et les acteurs sont énumérés en colonnes, avec leurs lignes de vie verticales indiquant la durée de vie de l'objet.

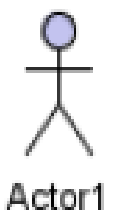
Objet

les objets sont des instances des classes, et sont rangés horizontalement. La représentation graphique pour un objet est similaire à une classe (un rectangle) précédée du nom d'objet (facultatif) et d'un point-virgule (:).



Acteur

les acteurs peuvent également communiquer avec des objets, ainsi ils peuvent eux aussi être énumérés en colonne. Un acteur est modélisé en utilisant le symbole habituel: *Stickman*.



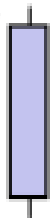
Représentation (2/2)

Ligne de vie

les lignes de vie, *LifeLine*, identifient l'existence de l'objet par rapport au temps. La notation utilisée pour une ligne de vie est une ligne pointillée verticale partant de l'objet.

Activation

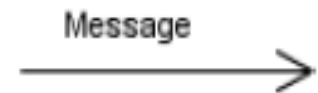
les activations, sont modélisées par des boîtes rectangulaires sur la ligne de vie. Elles indiquent quand l'objet effectue une action.



Un message définit une communication particulière entre des lignes de vie. Plusieurs types de messages existent

Message

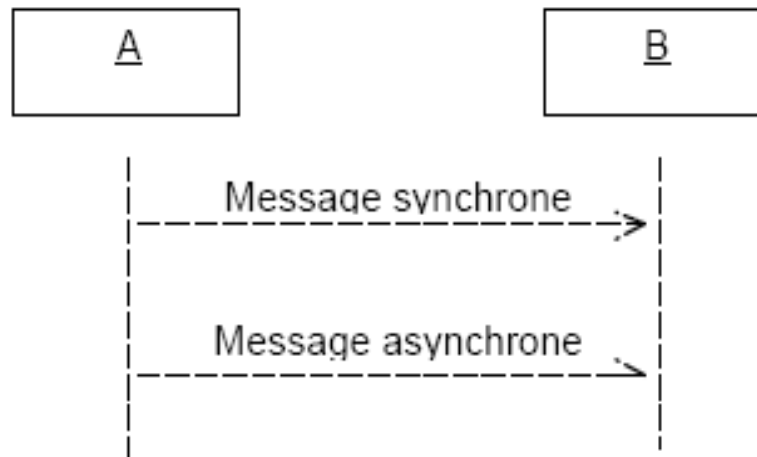
les messages, modélisés par des flèches horizontales entre les activations, indiquent les communications entre les objets.



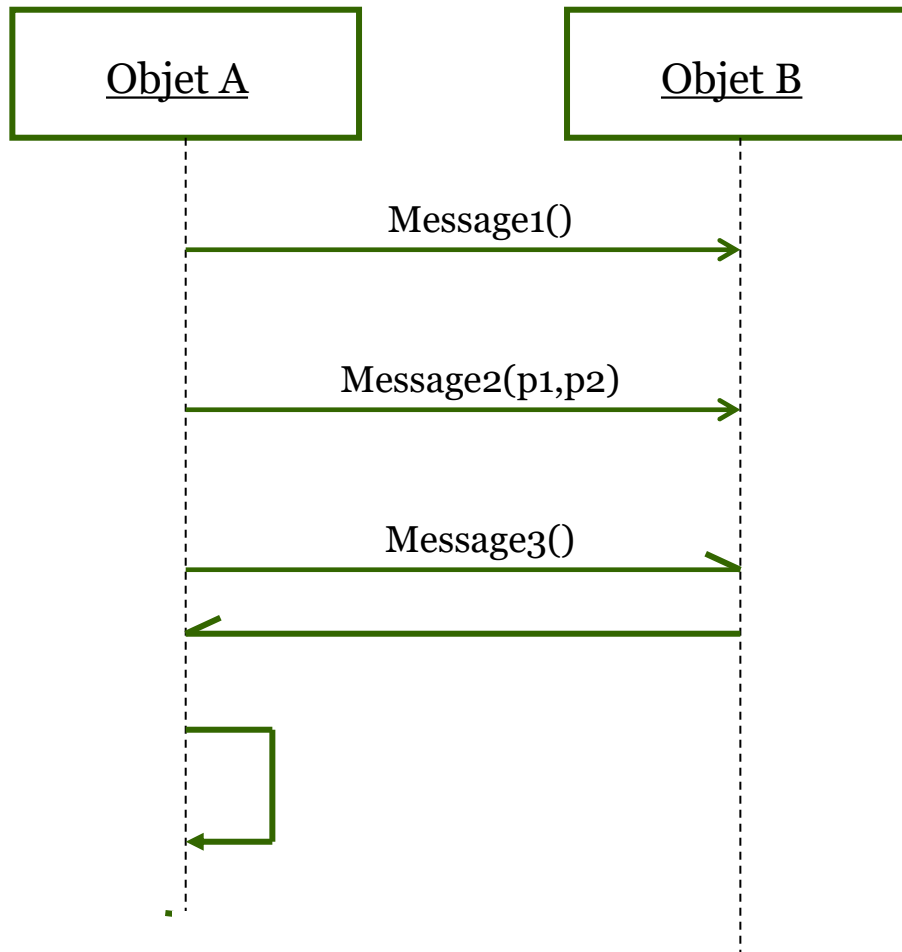
Les messages (1/2)

la séquence des messages échangés qui peuvent être : synchrone ou asynchrone

- les envois synchrones pour lesquels l'émetteur est bloqué et attend que l'appelé ait fini de traiter le message,
- les envois asynchrones pour lesquels l'émetteur n'est pas bloqué et peut continuer son exécution.



Les messages (2/2)



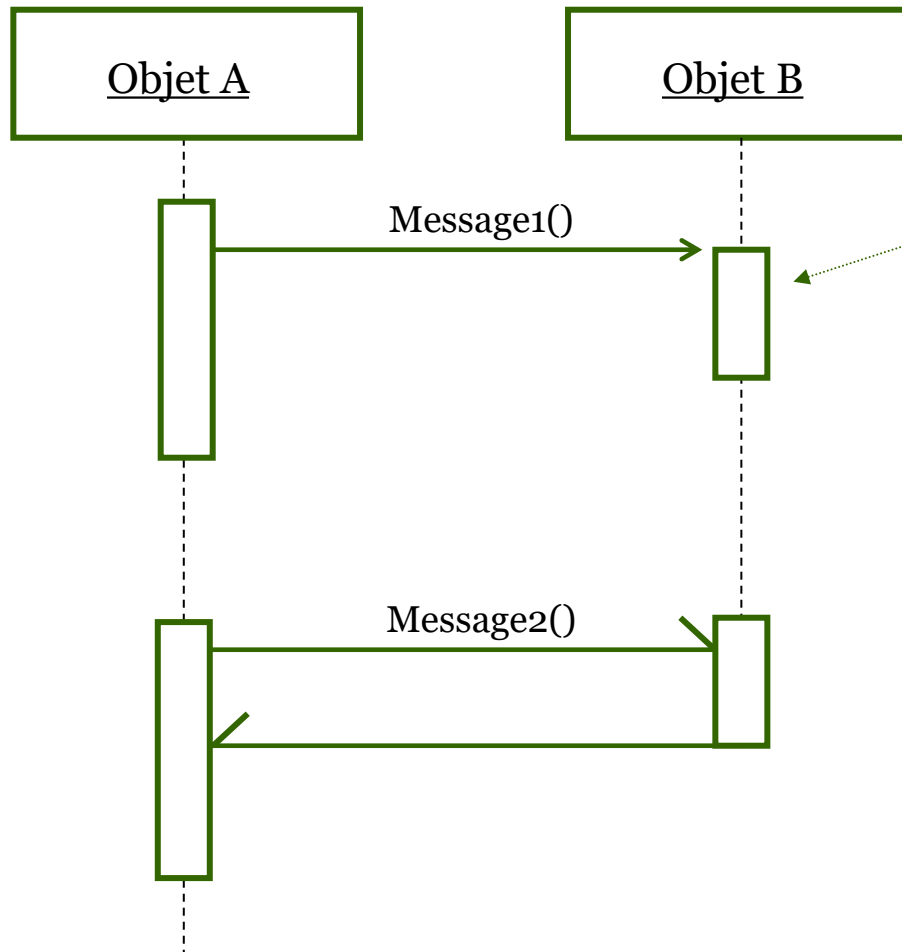
Le retour est implicite

Le message a des paramètres
Le retour est implicite

Le retour est explicite

Message récursif (objet composite)

Période d'activité



Période d'activité

l'objet A active l'objet B et reprendra la main quand B lui redonne la main

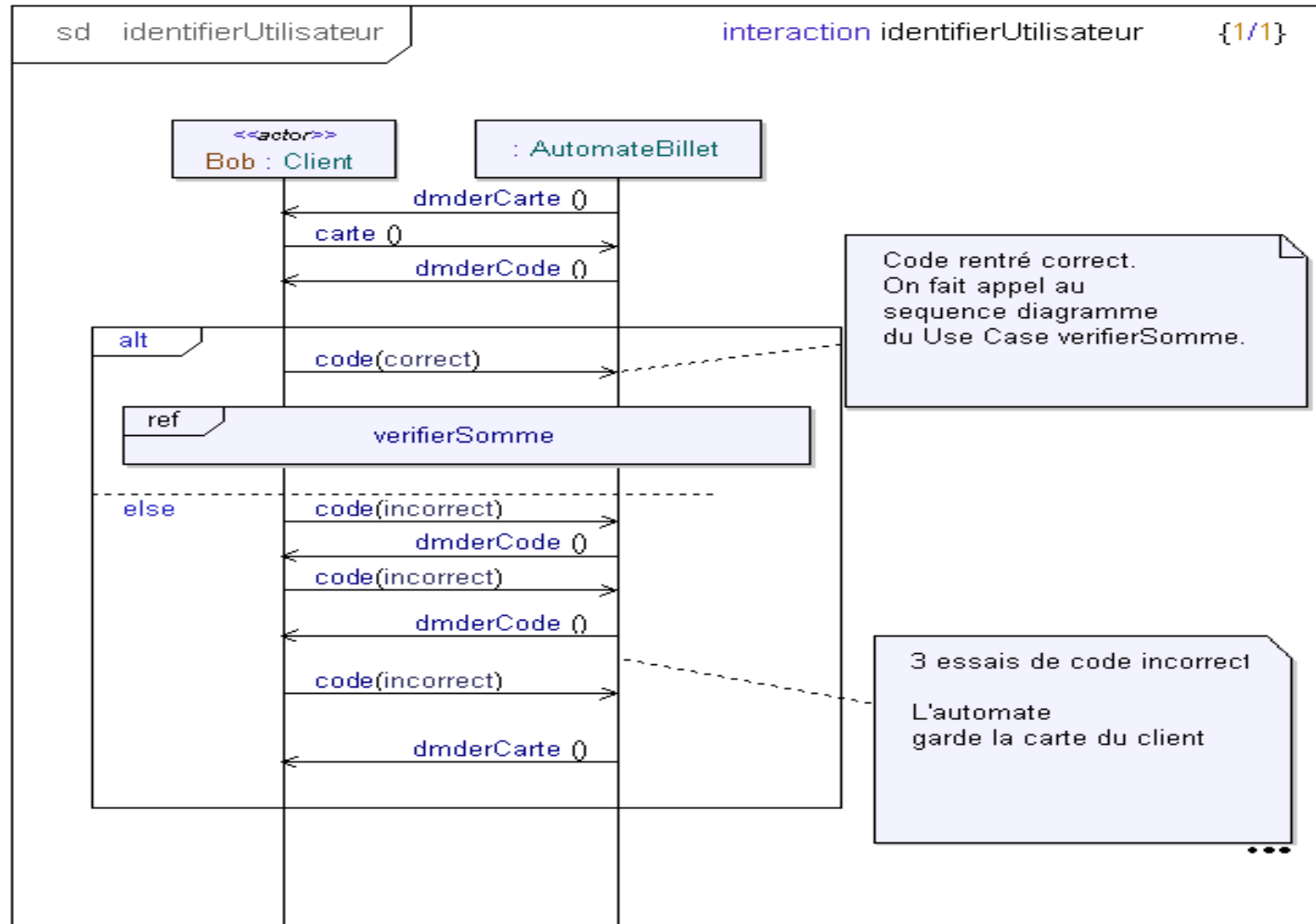
Le retour est explicite avant suicide :
la ligne de vie est terminée

Les opérateurs

L'opérateur alternative :

L'opérateur "**alt**" désigne un choix, une alternative. Il représente deux comportements possibles : c'est en quelque sorte l'équivalent du **SI...ALORS...SINON** : donc, une seule des deux branches sera réalisée dans un scénario donné. La condition d'exécution d'une des deux branches (l'équivalent du SI) peut être explicite ou implicite. L'utilisation de l'**opérateur else** permet d'indiquer que la branche est exécutée si la condition du alt est fausse.

L'opérateur alternative

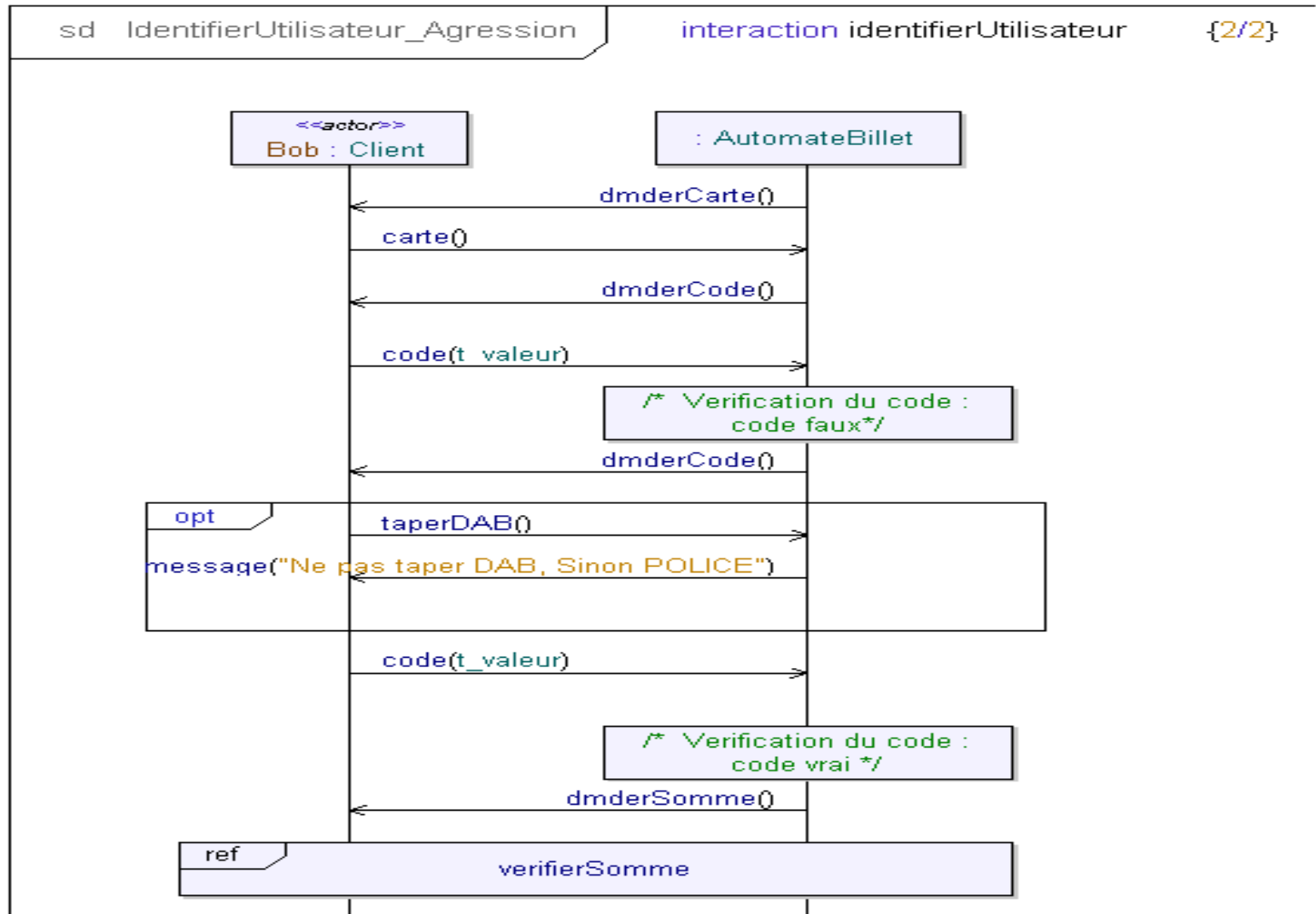


Les opérateurs

L'opérateur option :

L'opérateur "**opt**" désigne un fragment combiné optionnel comme son nom l'indique : c'est à dire qu'il représente un comportement qui peut se produire... ou pas. Un fragment optionnel est équivalent à un fragment "alt" qui ne posséderait pas d'opérande else (qui n'aurait qu'une seule branche). Un fragment optionnel est donc une sorte de **SI...ALORS**.

L'opérateur option

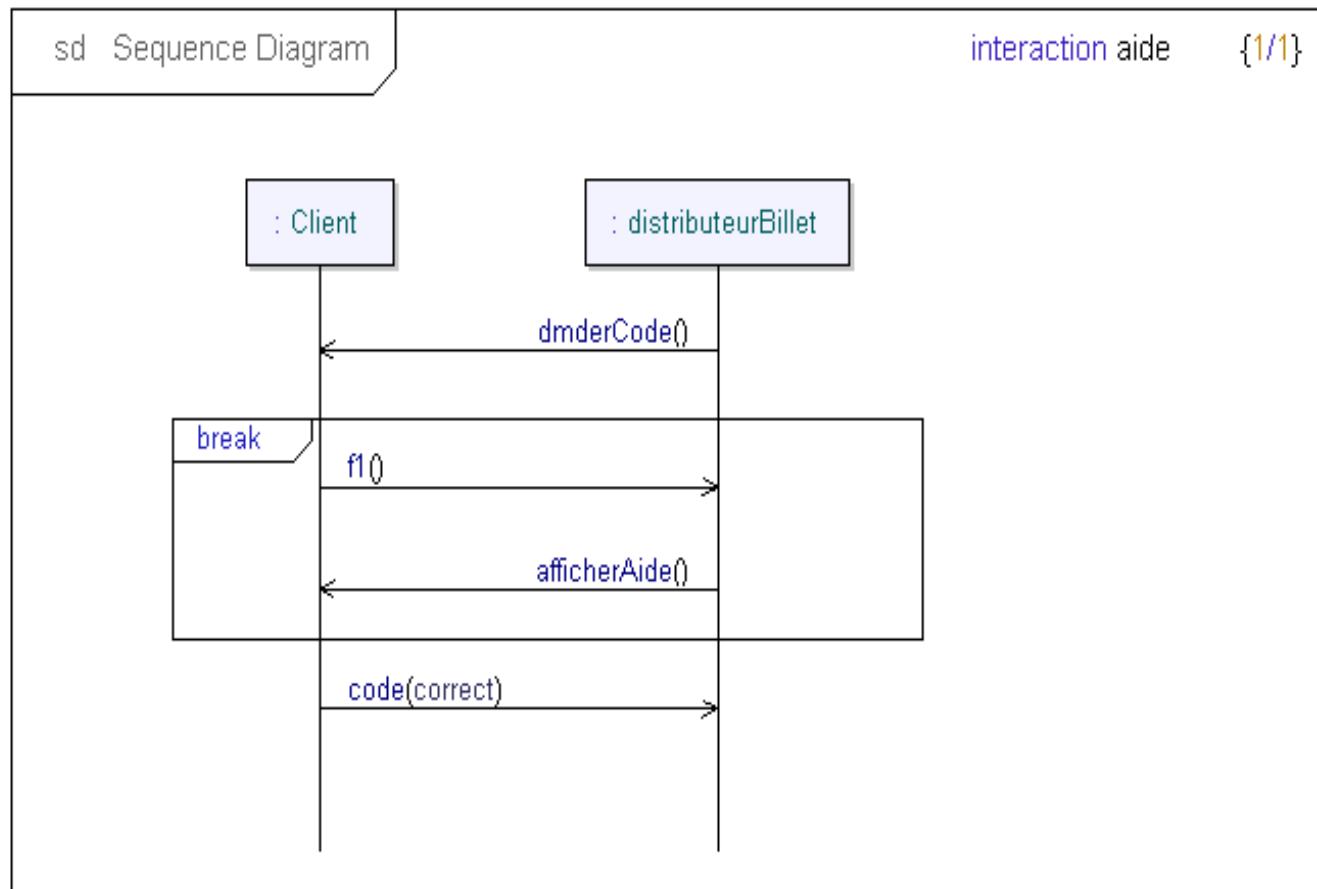


Les opérateurs

L'opérateur break:

L'opérateur "**break**" est utilisé dans les fragments combinés qui représentent des scénarii d'**exception** en quelque sorte. Les interactions de ce fragment seront exécutées à la place des interactions décrites en dessous. Il y a donc une notion d'interruption du flot "normal" des interactions.

L'opérateur break

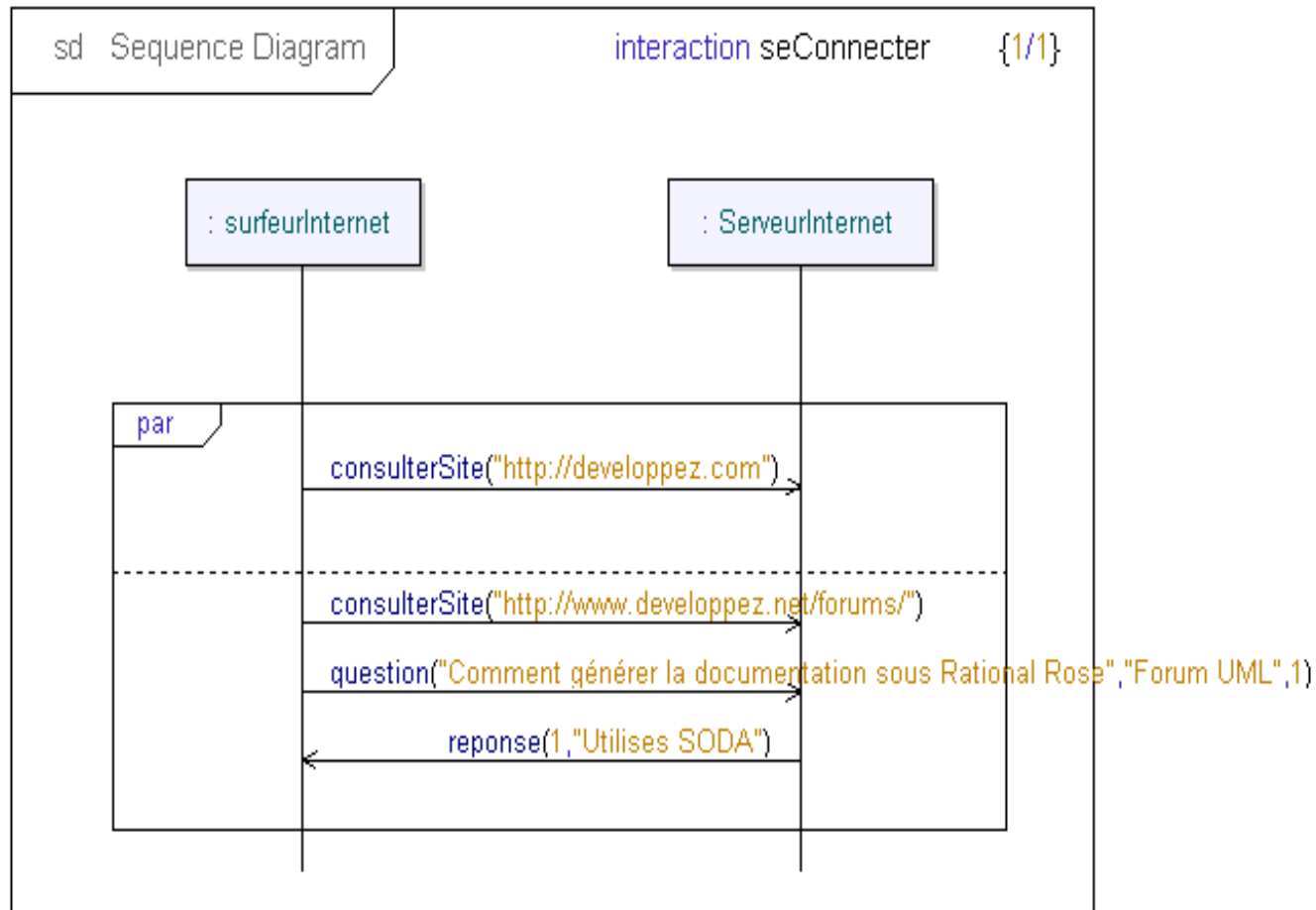


Les opérateurs

L'opérateur parallèle :

L'opérateur "**par**" est utilisé pour représenter des interactions ayant lieu en **parallèle**. Les interactions des différents opérandes (les deux branches de notre opérateur ci-dessous) peuvent donc se mélanger, s'intercaler, dans la mesure où l'ordre imposé dans chaque opérande est respecté.

L'opérateur parallèle

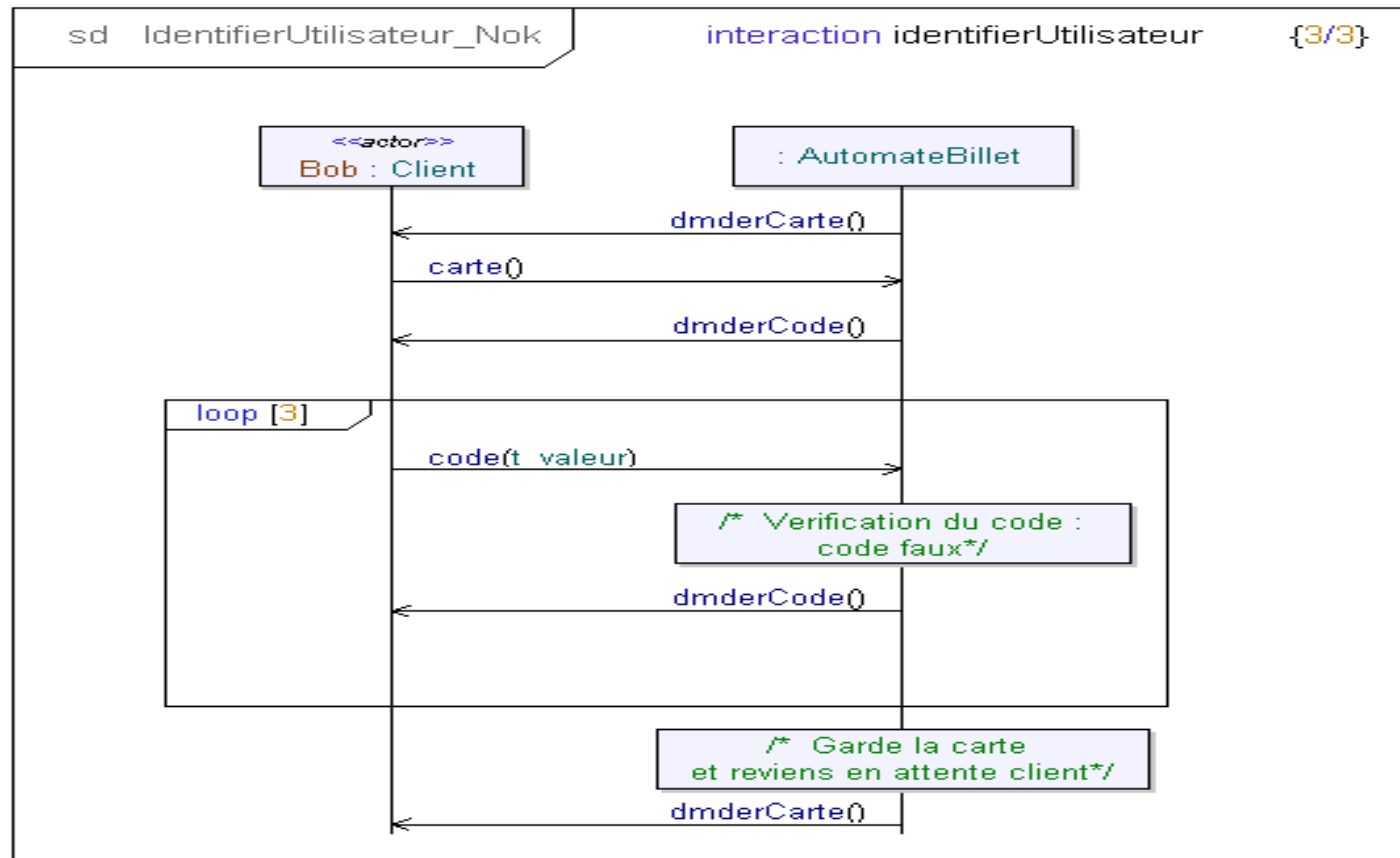


Les opérateurs

L'opérateur loop:

L'opérateur "**Loop**" (boucle) est noté "**loop**". Cet opérateur est utilisé pour décrire un ensemble d'interaction qui s'exécutent **en boucle**. En général, une contrainte appelée **garde** indique le nombre de répétitions (minimum et maximum) ou bien une condition booléenne à respecter.

L'opérateur loop



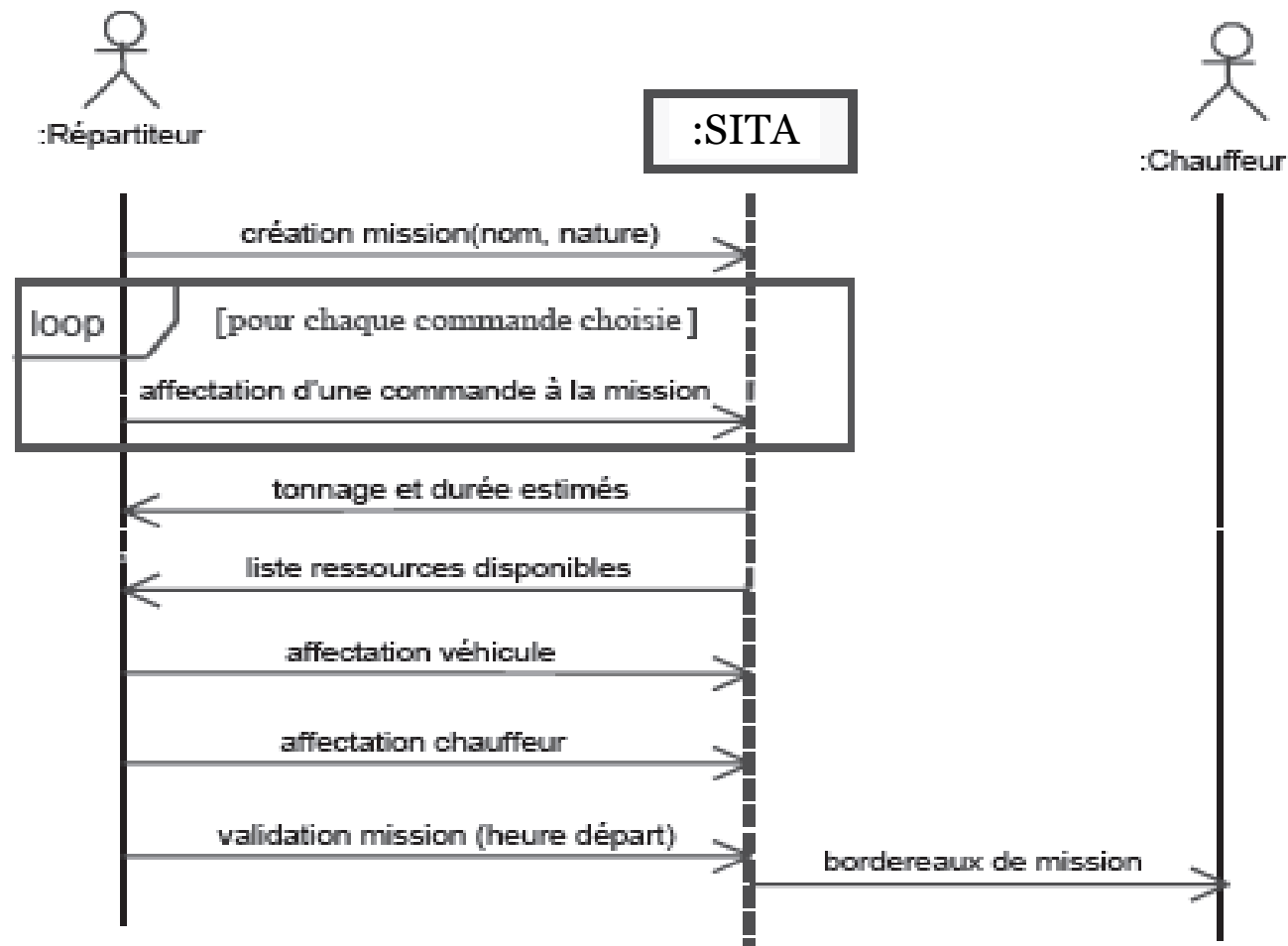


Figure 4-7 : Diagramme de séquence du scénario nominal du cas « Planifier une mission »

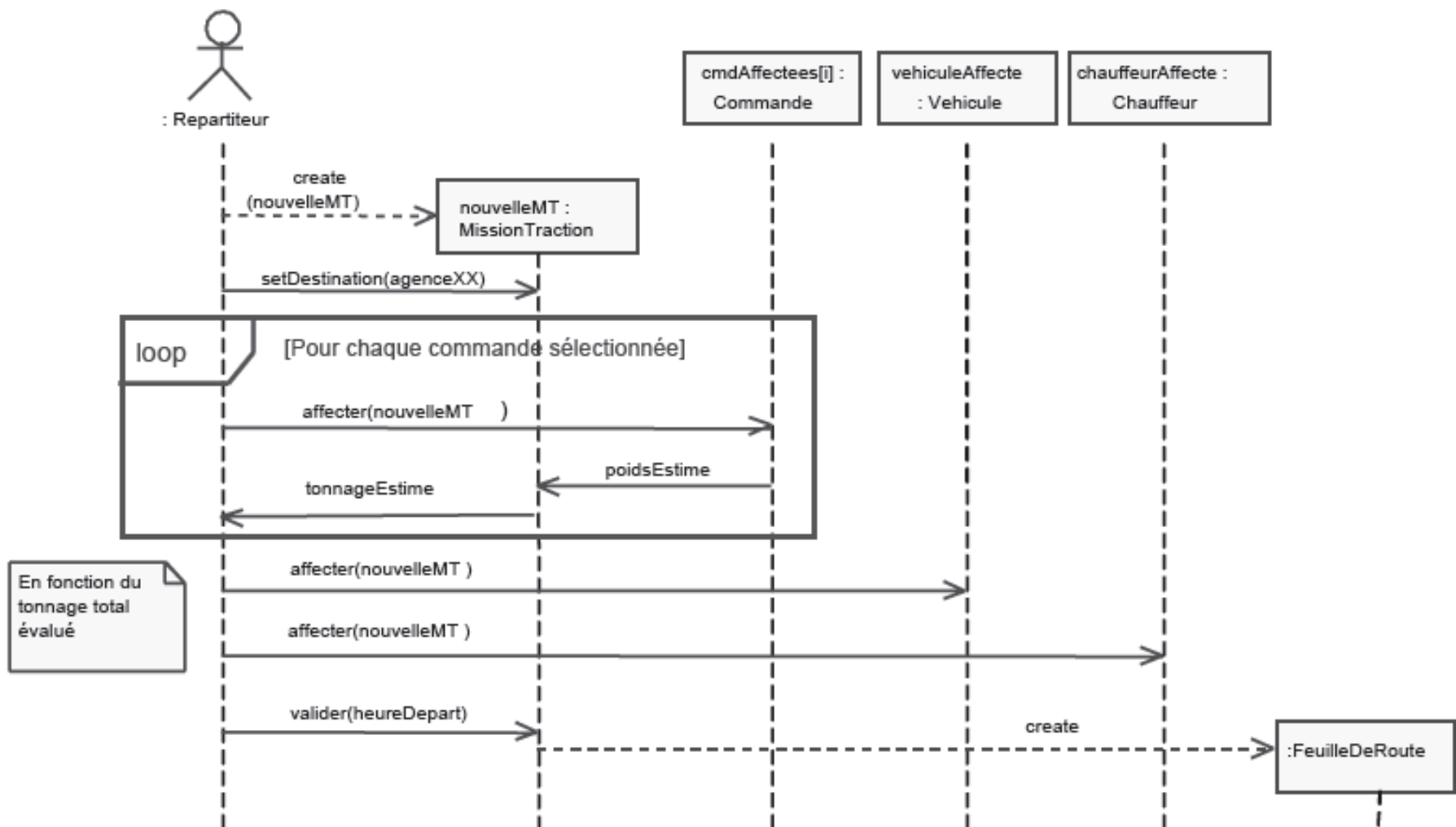


Figure 8-4 : Diagramme de séquence du scénario PM_N1

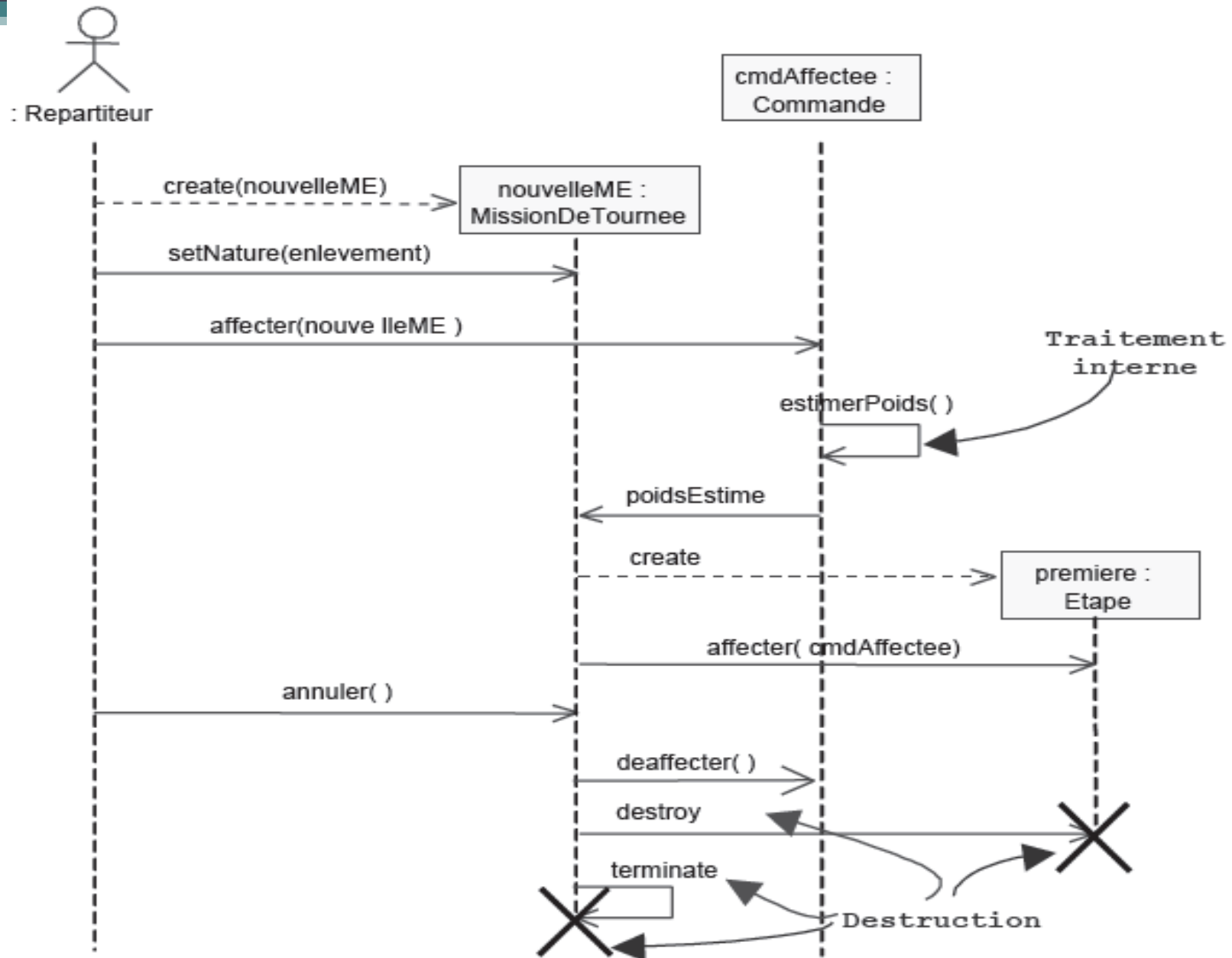


Figure 8-6 : Diagramme de séquence du scénario PM_N2

Diagramme de classe

Concepts de l'approche objet

Notion de classe

Une classe est un **type de données** abstrait, caractérisé par des propriétés (attributs et méthodes) communes à toute une famille d'objets et permettant de créer (instancier) des objets possédant ces propriétés.

Encapsulation

L'encapsulation consiste à masquer les détails d'implémentation d'un objet, en définissant une interface.

L'interface est la vue externe d'un objet, elle définit les services accessibles (offerts) aux utilisateurs de l'objet.

L'encapsulation facilite l'évolution d'une application car elle stabilise l'utilisation des objets : on peut modifier l'implémentation des attributs d'un objet sans modifier son interface, et donc la façon dont l'objet est utilisé.

L'encapsulation garantit l'intégrité des données, car elle permet d'interdire, ou de restreindre, l'accès direct aux attributs des objets.

Concepts de l'approche objet

Héritage (Spécialisation, Généralisation)

L'héritage est un mécanisme de transmission des propriétés d'une classe (ses attributs et méthodes) vers une sous-classe. Une classe peut être spécialisée en d'autres classes, afin d'y ajouter des caractéristiques spécifiques ou d'en adapter certaines. Plusieurs classes peuvent être généralisées en une classe qui les factorise, afin de regrouper les caractéristiques communes d'un ensemble de classes.

Ainsi, la spécialisation et la généralisation permettent de construire des hiérarchies de classes. L'héritage peut être simple ou multiple. L'héritage évite la duplication et encourage la réutilisation.

Polymorphisme

Le polymorphisme représente la faculté d'une méthode à pouvoir s'appliquer à des objets de classes différentes. Le polymorphisme augmente la généricité, et donc la qualité, du code.

Concepts de l'approche objet

Agrégation

Il s'agit d'une relation entre deux classes, spécifiant que les objets d'une classe sont des composants de l'autre classe.

Une relation d'agrégation permet donc de:

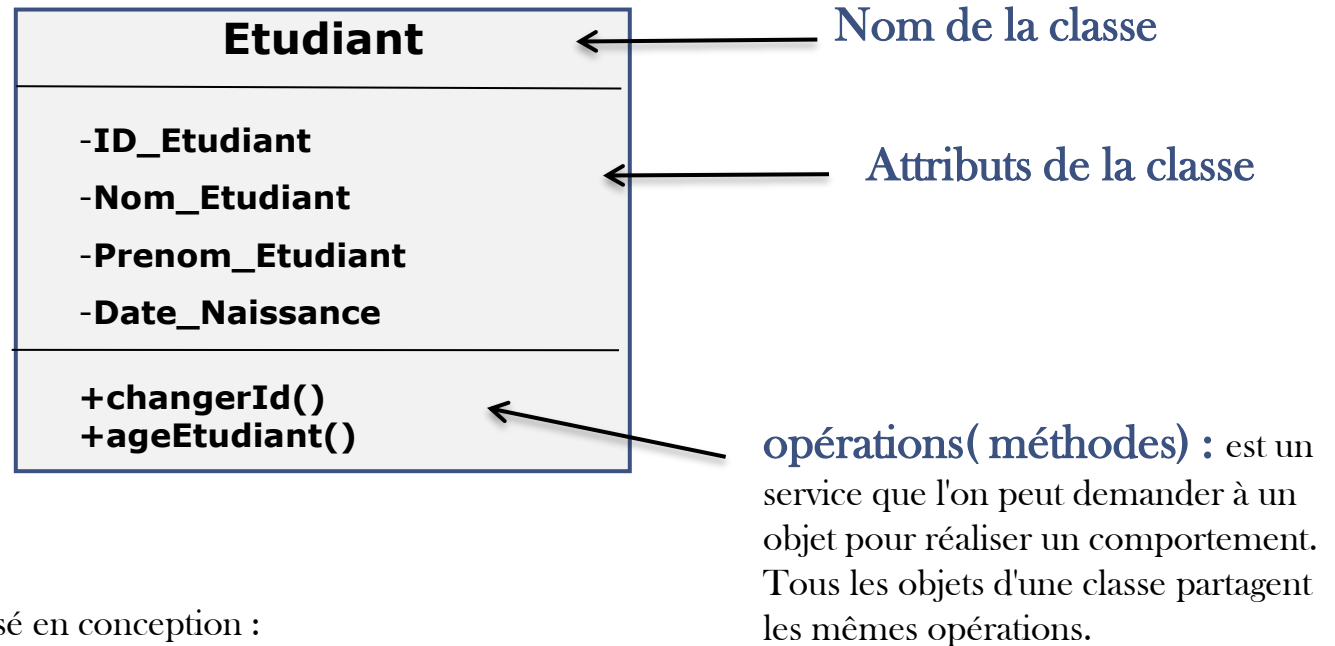
- Définir des objets composés d'autres objets.
- Assembler des objets de base, afin de construire des objets plus complexes.

Diagramme de classe

Diagramme de cas d'utilisation représente le système de **point de vue acteurs**, alors qu'il faut modéliser la structure interne du système: classe

Le diagramme de classe est un diagramme qui structure des données du système définies comme un ensemble de relations entre classes

Diagramme de classe : représentation graphique



Marqueur optionnel utilisé en conception :

- + public : accessible à tous
- # protégé : accessible à la classe et aux classes qui en héritent
- privé : accessible à la classe seule (valeur par défaut).
- Rien : accessible aux classes du même package.

Diagramme de classe : relations

Le diagramme de classes est modélisé par un ensemble de classes reliées entre-elles.

Les relations entre classes peuvent être:

- Une relation type **association**;
- une relation de type **agrégation** et **composition**;
- une relation de type héritage : **généralisation** et **spécialisation**.

Diagramme de classe : relation d'association

Une association exprime une connexion sémantique bidirectionnelle entre plusieurs classes(Majorité des cas entre deux classes) : Association binaire

Pour améliorer la lisibilité, l'association peut être nommée par une forme verbale. Le sens de la lecture peut être précisé



Diagramme de classe : relation d'association

Une association est décrite par:

1. Nom de l'association:

- ✓ Commence généralement par un majuscule
- ✓ Significatif de la relation

2. Le rôle :

- ✓ Le *nom de rôle* est un nom qui identifie de façon unique une extrémité de l'association.

3. La multiplicité ou cardinalité:

- ✓ La *multiplicité* précise combien d'instances d'une classe peuvent se rattacher à une seule instance d'une classe donnée.
- ✓ La notation générale adoptée est : **min** . . **Max**

Diagramme de classe : relation d'association

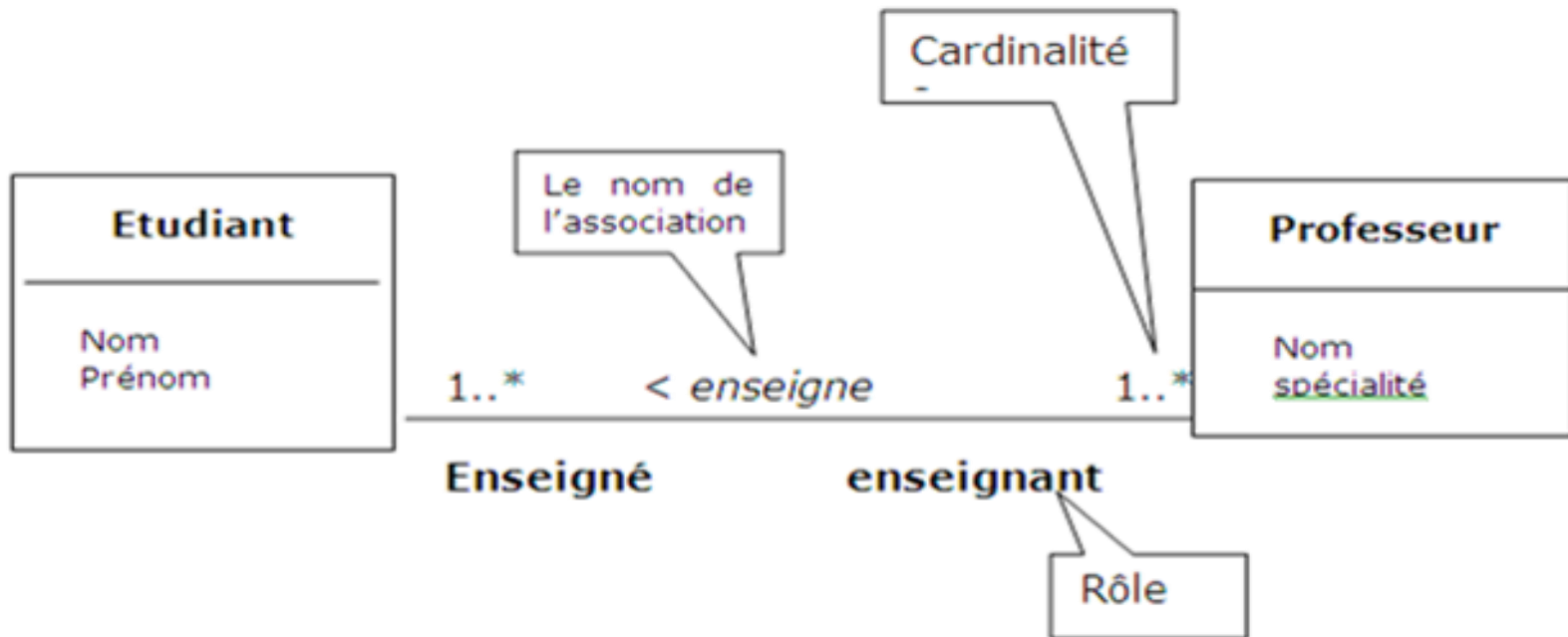
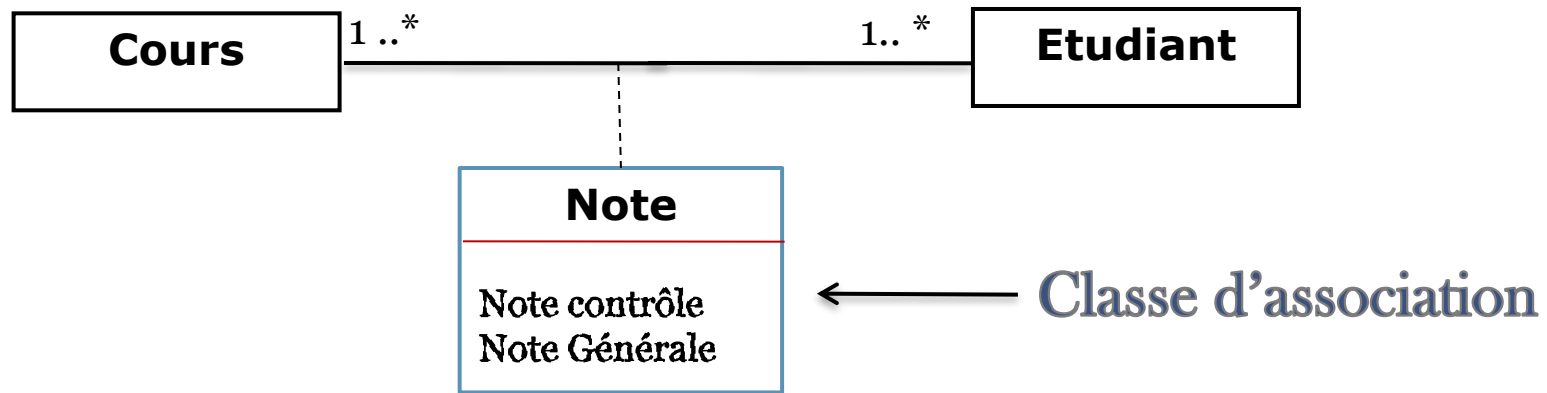


Diagramme de classe : types de relation d'association

Classe d'association : il s'agit d'une classe qui réalise la navigation entre les instances d'autres classes (Association est représentée par une classe)



Réflexive : Une association réflexive se traduit simplement par une référence sur un objet de la même classe.

Diagramme de classe : agrégation et composition

L'agrégation : une forme d'association forte dans laquelle un objet agrégat (Composé) est fait de composants. Les composants font partie de l'agrégat.

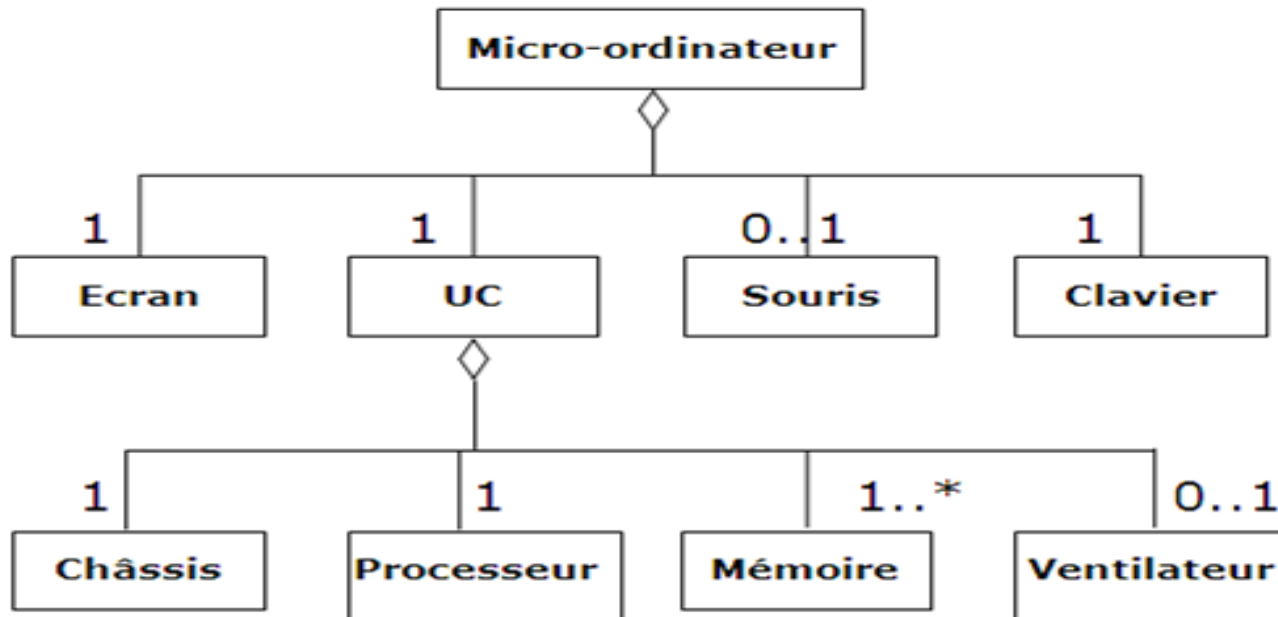


Diagramme de classe : agrégation et composition

Une composition : une forme forte d'agrégation dans laquelle le cycle de vie des parties composantes est lié à celui du composé. ils vivent et meurent avec lui.

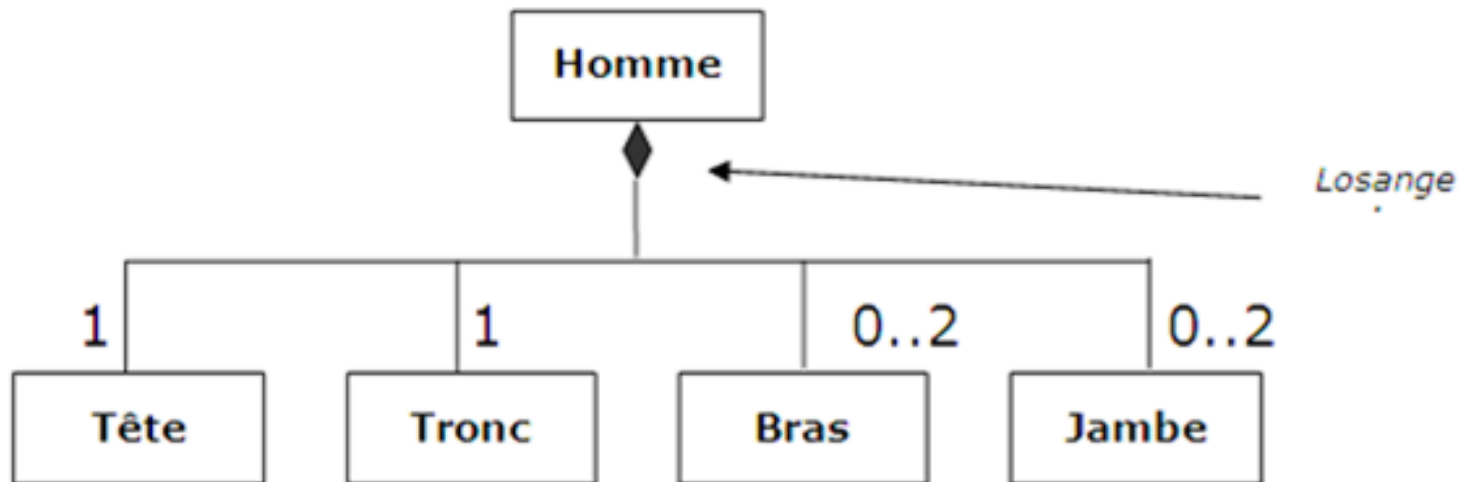


Diagramme de classe : héritage

Les hiérarchies de classes permettent de gérer la complexité, en ordonnant les objets au sein d'arborescences de classes.

Généralisation:

- Démarche ascendante, qui consiste à capturer les particularités communes d'un ensemble d'objets, issus de classes différentes.
- Consiste à factoriser les propriétés d'un ensemble de classes, sous forme d'une super-classe, plus abstraite.

Spécialisation:

- Démarche descendante, qui consiste à étendre les propriétés des classes sous forme de sous-classes plus spécifiques.

Diagramme de classe : héritage

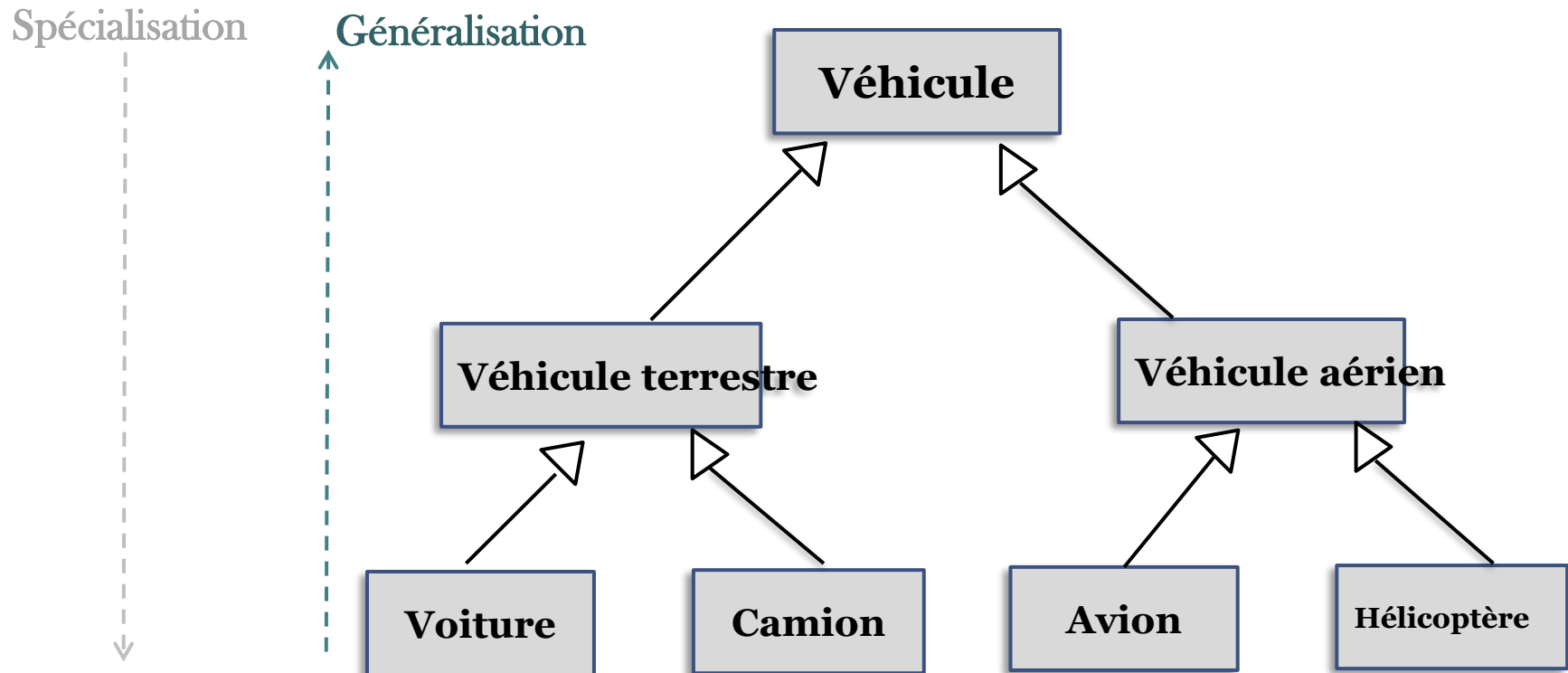
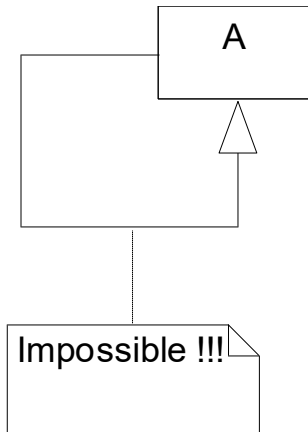
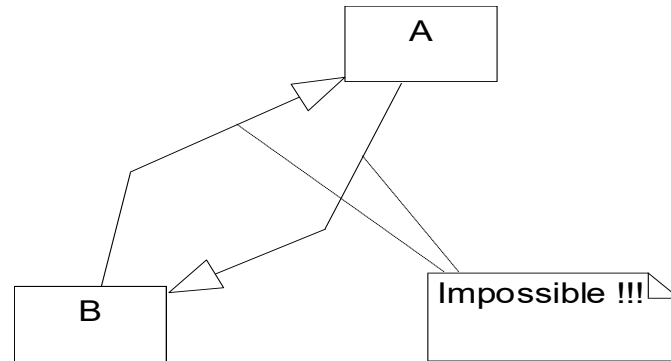


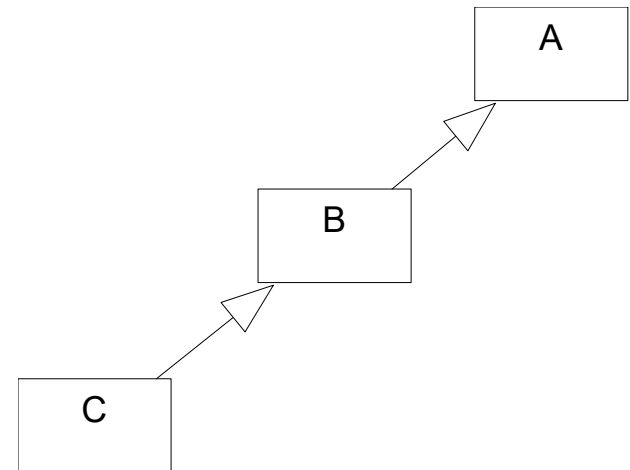
Diagramme de classe : propriétés de la généralisation



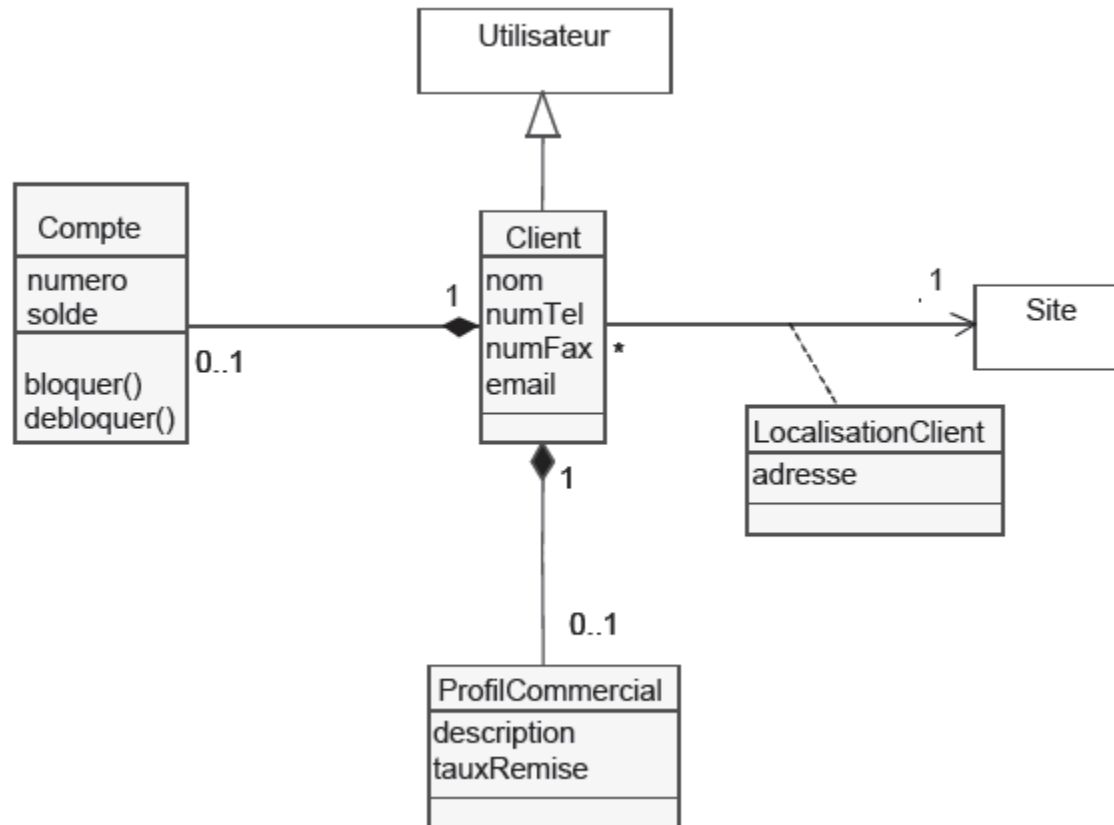
Non réflexive



Non symétrique



Transitive



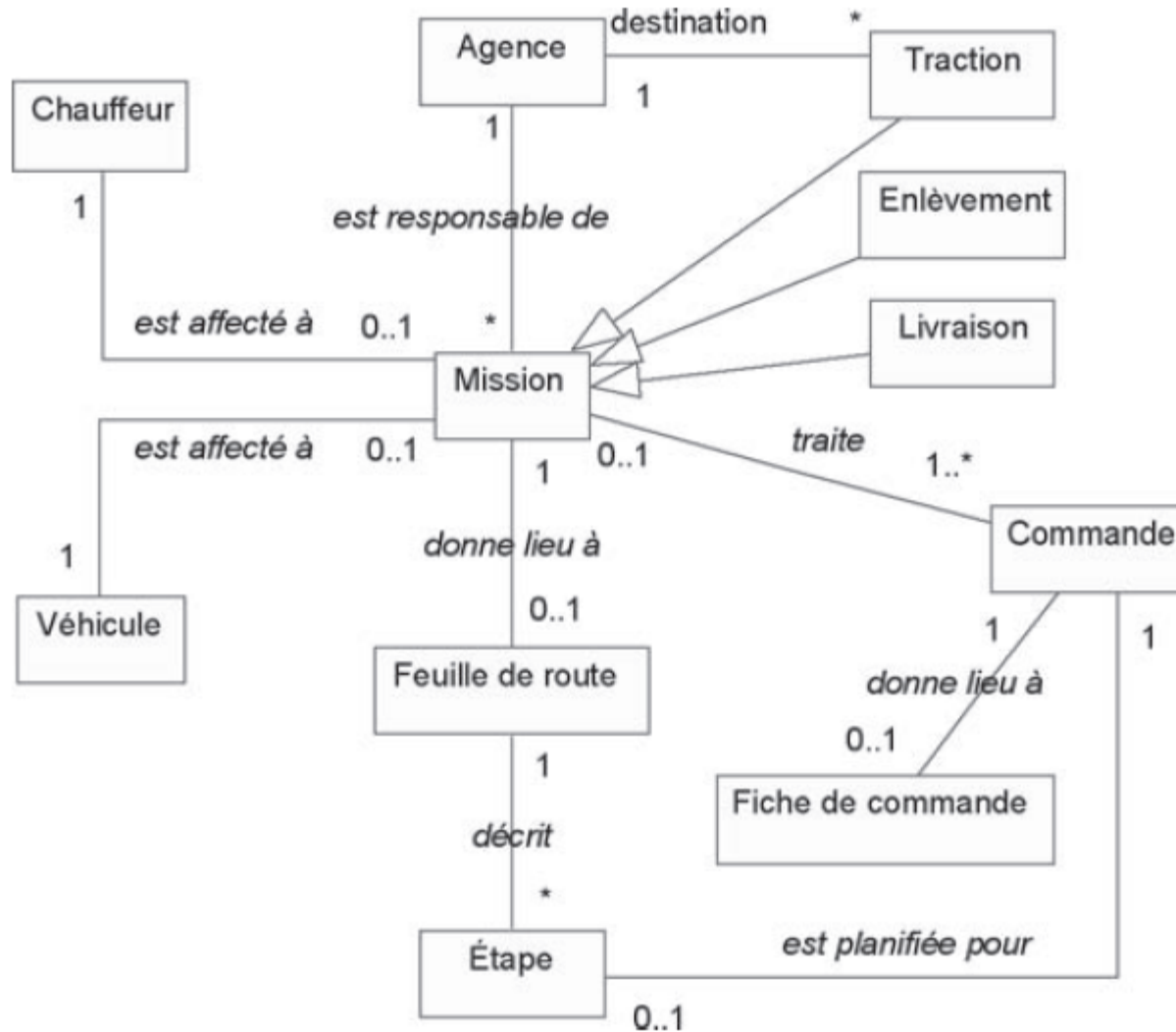


Diagramme des classes participantes complété de « Planifier une mission »

Diagramme d'état

Diagramme d'état

Les diagrammes d'états-transitions d'UML décrivent le comportement interne d'un objet à l'aide d'un automate à états finis.

Ce diagramme est utilisé pour montrer l'historique de la vie d'une classe donnée : des événements font passer d'un état à un autre état qui doit déclencher des actions.

C'est une abstraction des comportements possibles.

Diagramme d'état

Un diagramme d'états-transitions rassemble et organise les états et les transitions d'une classe donné.

Un diagramme d'états-transitions ne peut être associé qu'à une seule classe.

Diagramme d'état : notation

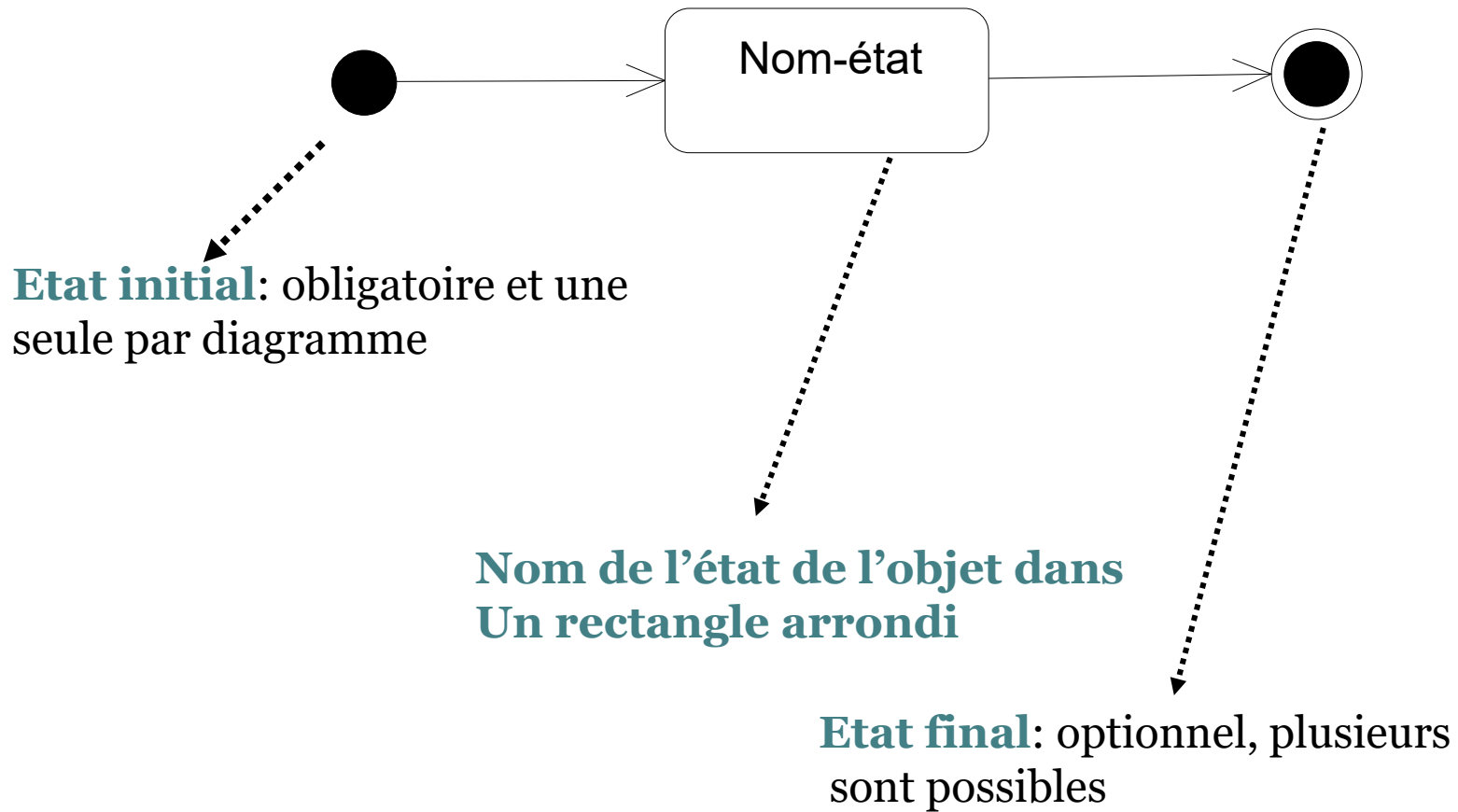


Diagramme d'état : Etat et Attributs

- ✓ Les états d'un objet d'une classe peuvent être distingués par les valeurs de certains attributs de l'objet
- ✓ Un seul état initial est possible
- ✓ Plusieurs états finaux

Diagramme d'état : Evénements

Un événement est une occurrence d'une situation particulière dans le domaine du problème.

L'état en cours de l'objet détermine la réaction de l'objet.

Quand un événement est reçu, une transition peut être déclenchée et faire basculer l'objet dans un nouvel état.

On peut diviser les événements en plusieurs types explicites et implicites : signal, appel, changement et temporel.

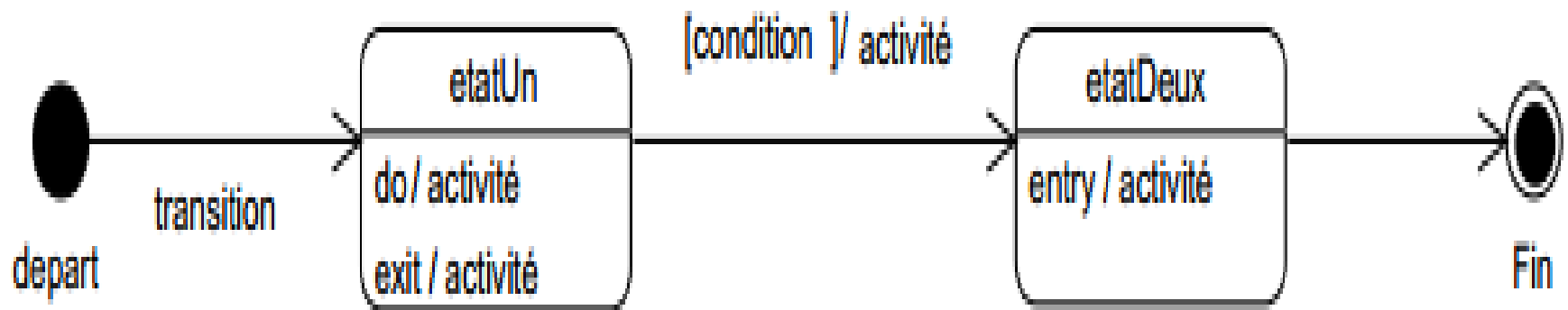


Diagramme d'état : Transitions

Une **transition** définit la réponse d'un objet à l'occurrence d'un événement. Elle lie, généralement, deux états E1 et E2 et indique qu'un objet dans un état E1 peut entrer dans l'état E2 et exécuter certaines activités, si un événement déclencheur se produit et que la condition de garde est vérifiée.

La syntaxe d'une transition est la suivante :

[<événement>] ['[' <garde> ']'] ['/' <activité>]

Le même événement peut être le déclencheur de plusieurs transitions quittant un même état. Chaque transition avec le même événement **doit avoir une condition de garde différente**. En effet, une seule transition peut se déclencher dans un même flot d'exécution. Si deux transitions sont activées en même temps par un même événement, une seule se déclenche et le choix n'est pas prévisible (*i.e.* pas déterministe).

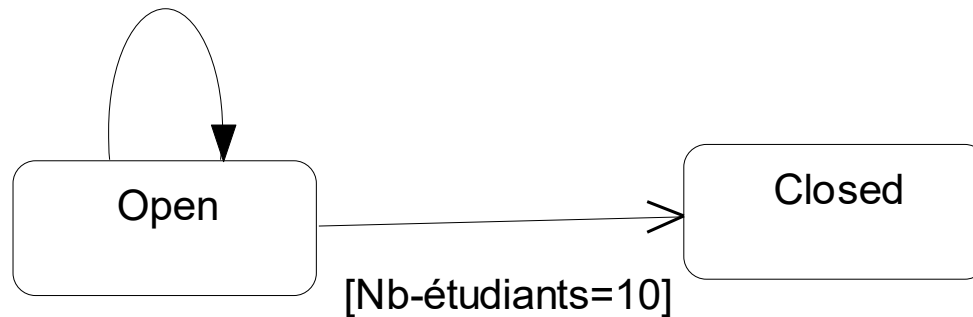
Diagramme d'état : Transitions et condition de garde

Une transition peut avoir une condition de garde (spécifiée par '[' <garde> ']' dans la syntaxe).

Il s'agit d'une expression logique sur les attributs de l'objet

Diagramme d'état : Transition (exemple)

Ajouter-étudiants[Nb-étudiants<10]



Condition de déclenchement de la transition: **Garde**

Diagramme d'état : Exemple 1

Voici un exemple de diagramme d'état qui modélise l'état du compte d'un utilisateur:

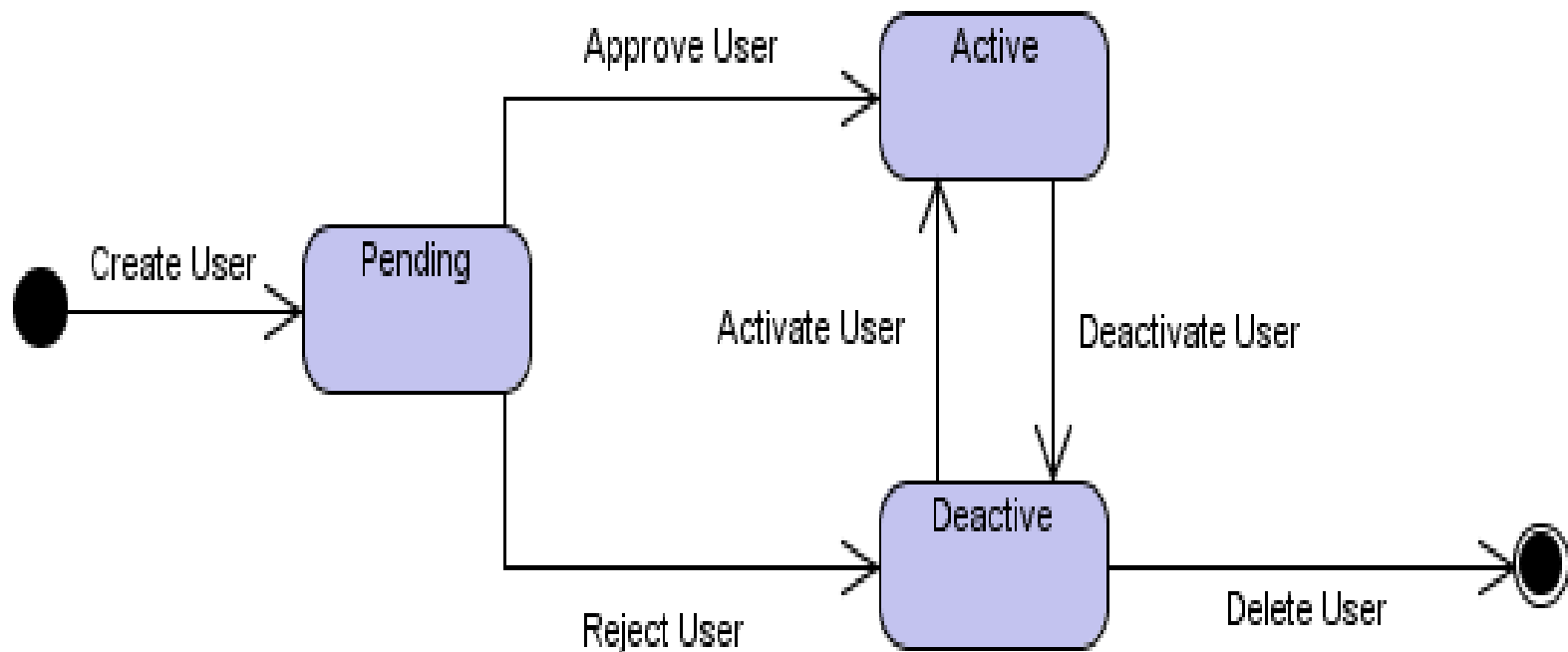


Diagramme d'état : Exemple 2

Gestion d'une médiathèque : Classe document

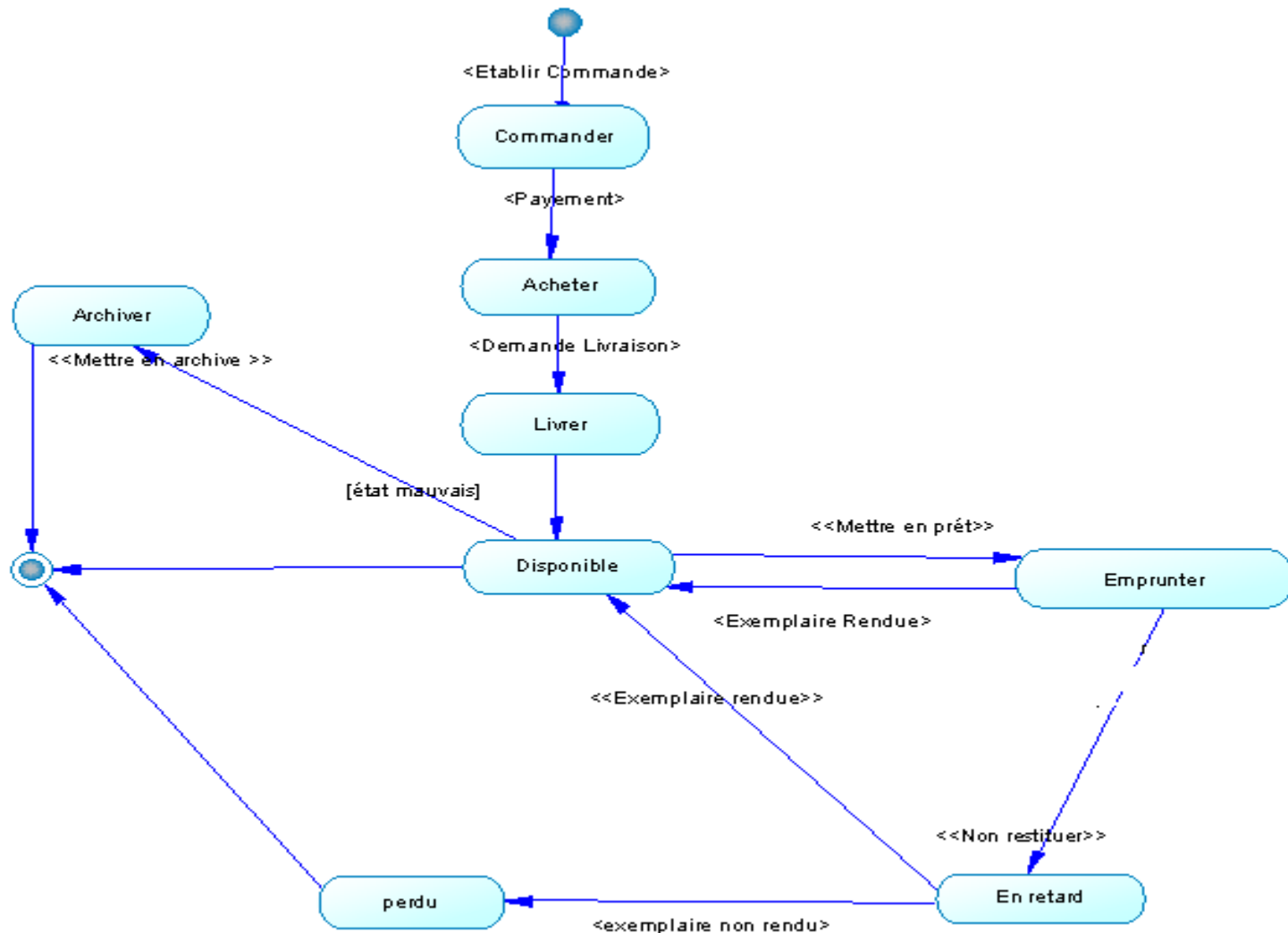


Diagramme d'activité

Diagramme d'activité

Les diagrammes d'activités permettent de mettre l'accent sur les traitements. Ils sont donc particulièrement adaptés à la modélisation du cheminement de flots de contrôle et de flots de données. Ils permettent ainsi de représenter graphiquement le comportement d'une méthode ou le déroulement d'un cas d'utilisation.

Les diagrammes d'activités sont relativement proches des diagrammes d'états-transitions dans leur présentation, mais leur interprétation est sensiblement différente.

La différence principale entre les diagrammes d'interaction et les diagrammes d'activités est que les premiers mettent l'accent sur le flot de contrôle d'un objet à l'autre, tandis que les seconds insistent sur le flot de contrôle d'une activité à l'autre.

Diagramme d'activité : Activité et transition

Une action est le plus petit traitement qui puisse être exprimé en UML.

Une action a une incidence sur l'état du système ou en extrait une information. Les actions sont des étapes discrètes à partir desquelles se construisent les comportements. La notion d'action est à rapprocher de la notion d'instruction élémentaire d'un langage de programmation (comme C++ ou Java). Une action peut être, par exemple :

- ✓ une affectation de valeur à des attributs ;
- ✓ un accès à la valeur d'une propriété structurelle (attribut ou terminaison d'association) ;
- ✓ la création d'un nouvel objet ou lien ;
- ✓ un calcul arithmétique simple ;
- ✓ l'émission d'un signal ;
- ✓ la réception d'un signal ;
- ✓ ...

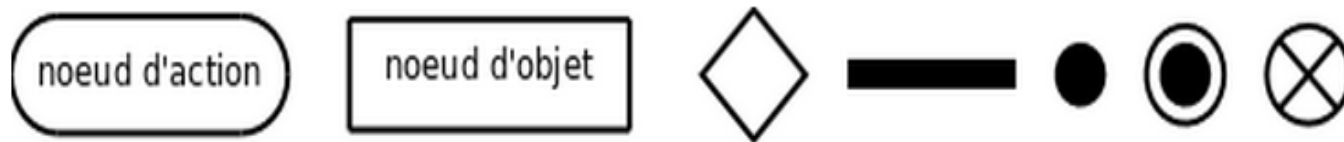
Diagramme d'activité : l'activité

Une activité définit un comportement décrit par un séquençement organisé d'unités dont les éléments simples sont les actions. Le flot d'exécution est modélisé par des nœuds reliés par des arcs (transitions). Le flot de contrôle reste dans l'activité jusqu'à ce que les traitements soient terminés.

Groupe d'activités (*activity group*)

Un groupe d'activités est une activité regroupant des nœuds et des arcs. Les nœuds et les arcs peuvent appartenir à plus d'un groupe. Un diagramme d'activités est lui-même un groupe d'activités.

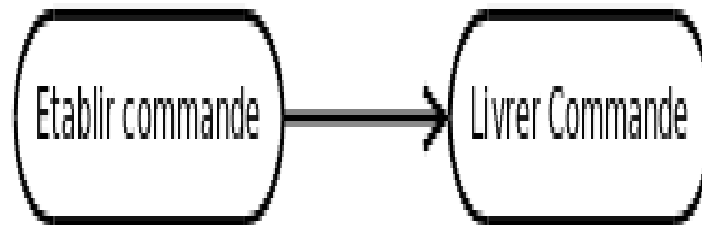
Nœud d'activité (*activity node*)



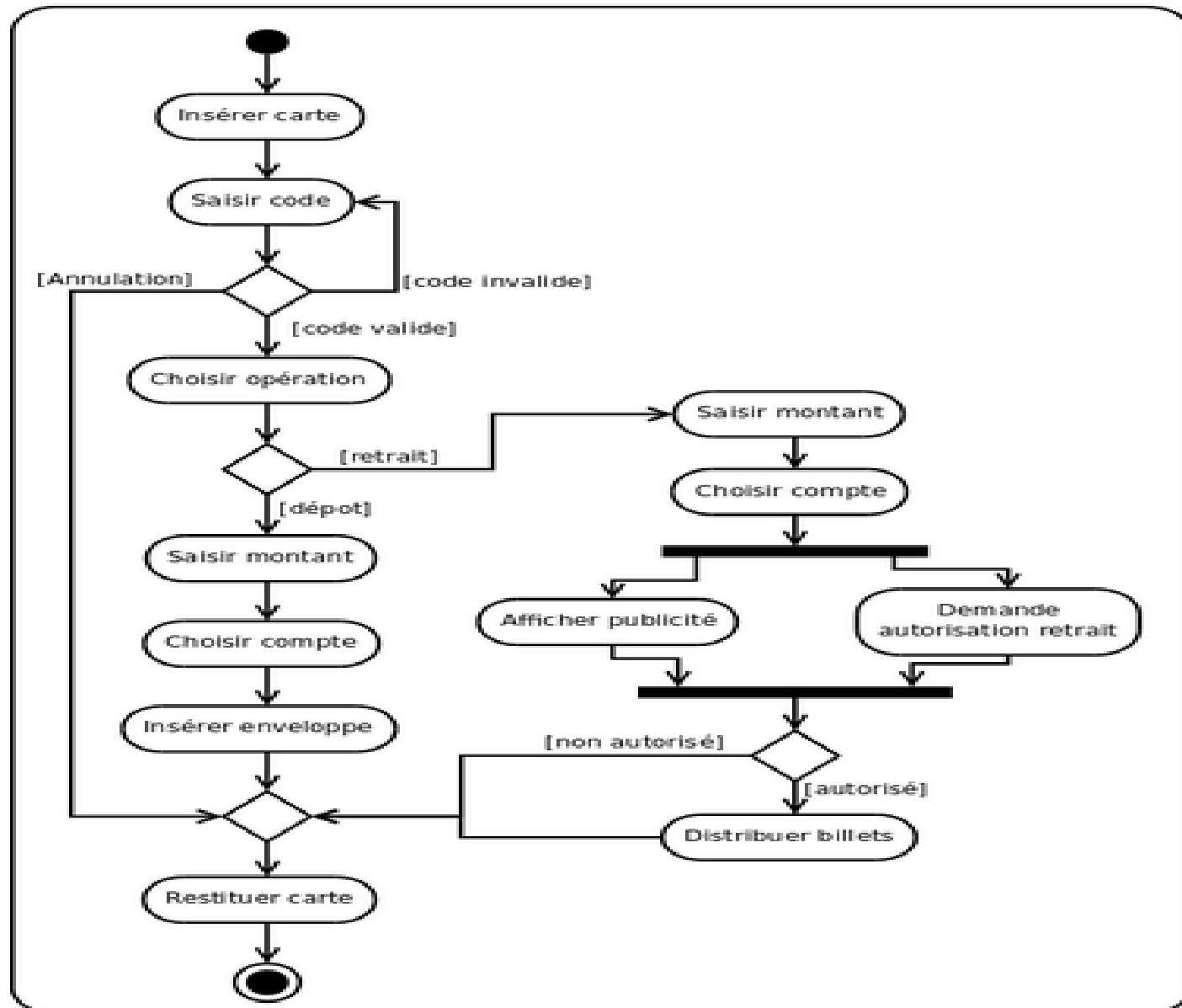
Représentation graphique des nœuds d'activité. De la gauche vers la droite, on trouve : le nœud représentant une action, qui est une variété de nœud exécutable, un nœud objet, un nœud de décision ou de fusion, un nœud de bifurcation ou d'union, un nœud initial, un nœud final et un nœud final de flot.

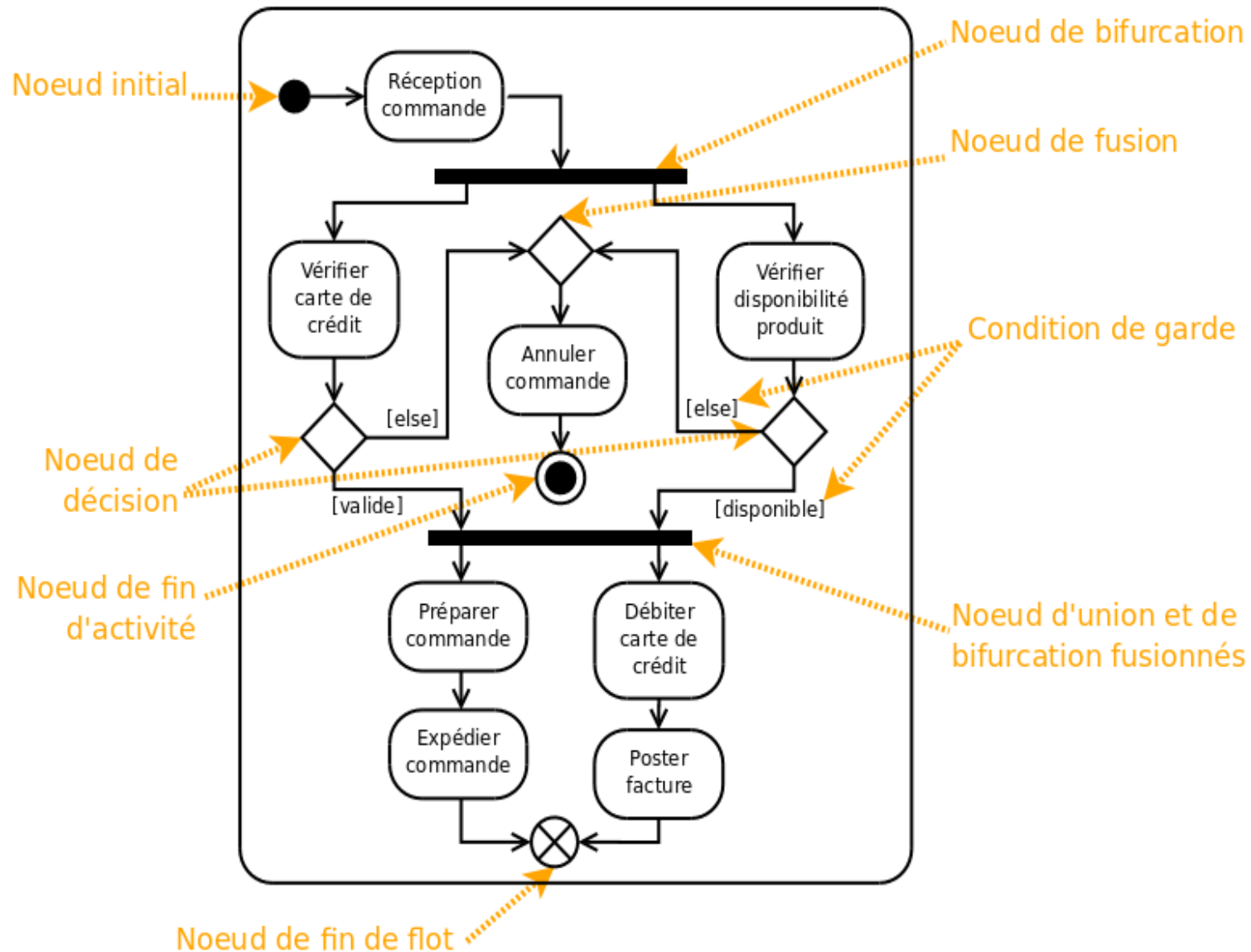
Diagramme d'activité : les transitions

Le passage d'une activité vers une autre est matérialisé par une transition. Graphiquement les transitions sont représentées par des flèches en traits pleins qui connectent les activités entre elles. Elles sont déclenchées dès que l'activité source est terminée et provoquent automatiquement et immédiatement le début de la prochaine activité à déclencher (l'activité cible). Contrairement aux activités, les transitions sont franchies de manière atomique, en principe sans durée perceptible.



Représentation graphique d'une transition

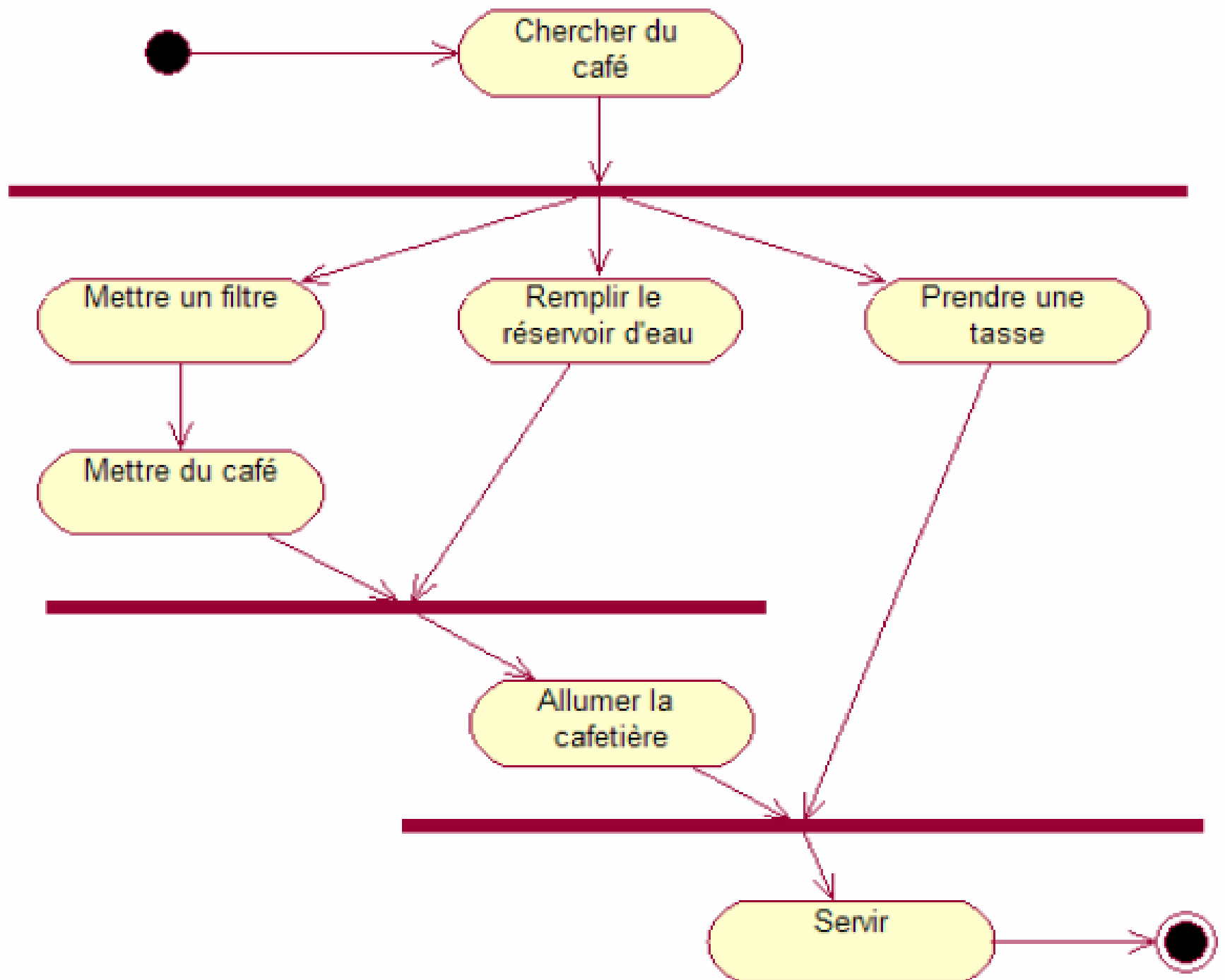




Exercice 1:

Construire un diagramme d'activité représentant l'utilisation d'une cafetière électrique :

- premier état : chercher du café
- dernier état : Servir du café



Exercice 2:

Construire un diagramme d'activité pour modéliser le processus de commander un produit. Le processus concerne les acteurs suivants :

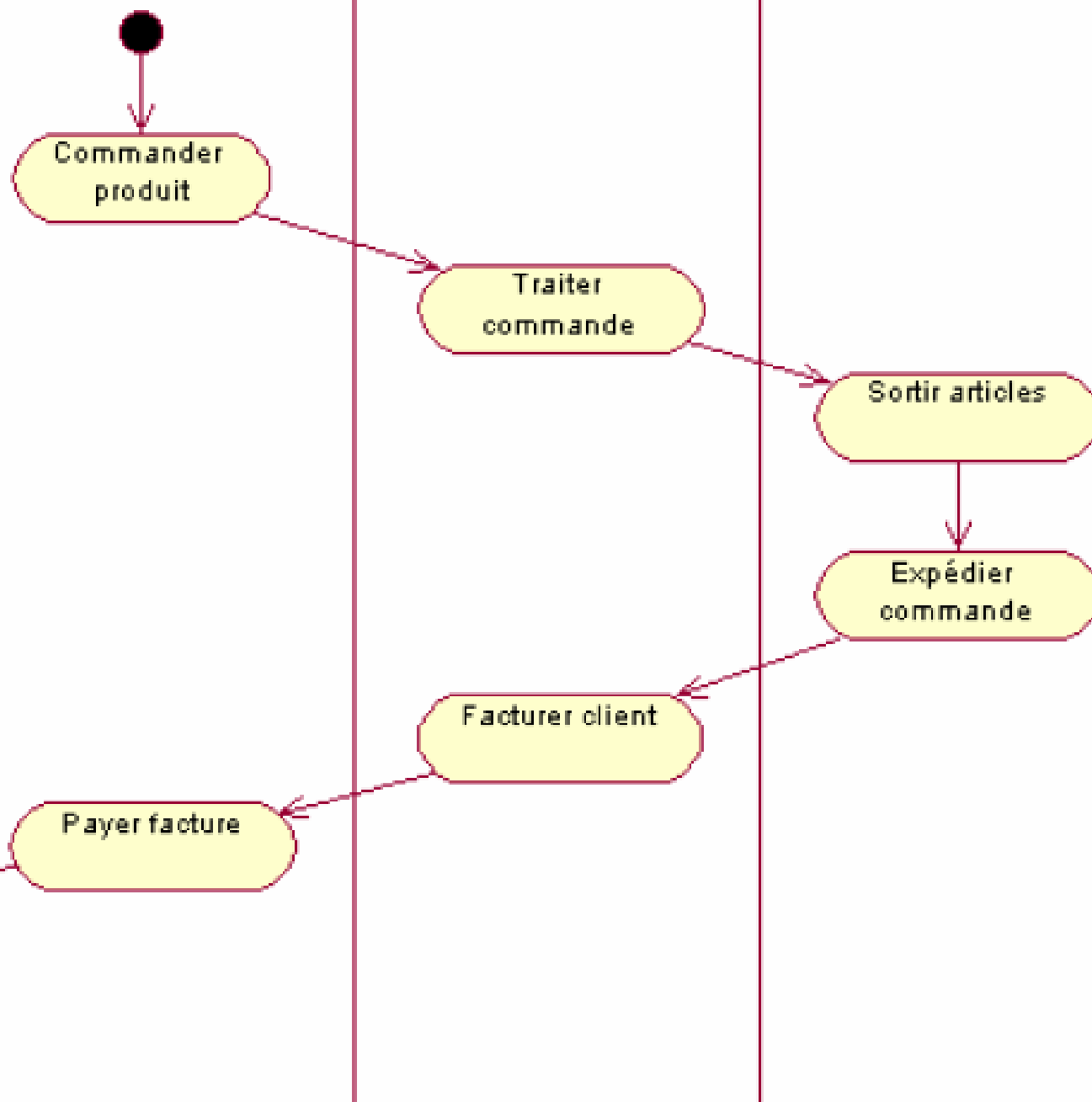
- **Client** : qui commande un produit et qui paie la facture
- **Caisse** : qui encaisse l'argent du client
- **Vente**: qui s'occupe de traiter et de facturer la commande du client
- **Entrepôt** : qui est responsable de sortir les articles et d'expédier la commande.

Caisse

: Client

Vente

Entrepot



Encaisser

Payer facture

Facturer client

Sortir articles

Expédier commande

Traiter commande

Commander produit

Exercice 3:

Le logiciel de gestion des réparations est destiné en priorité au chef d'atelier, il devra lui permettre de saisir les fiches de réparations et le travail effectué par les divers employés de l'atelier.

Pour effectuer leur travail, les mécaniciens et autres employés de l'atelier vont chercher des pièces de rechange au magasin. Lorsque le logiciel sera installé, les magasiniers ne fourniront des pièces que pour les véhicules pour lesquels une fiche de réparation est ouverte ; ils saisiront directement les pièces fournies depuis un terminal installé au magasin.

Lorsqu'une réparation est terminée, le chef d'atelier va essayer la voiture. Si tout est en ordre, il met la voiture sur le parc clientèle et bouclera la fiche de réparation informatisée. Les fiches de réparations bouclées par le chef d'atelier devront pouvoir être importées par le comptable dans le logiciel comptable.

1. Créer un diagramme d'activité pour tout le traitement d'une réparation.

